

DSA Cheatsheet — Tackle Every Topic With High Confidence

144 problems across 29 patterns, grouped into 4 tiers. For each topic: the mental model that makes you *recognize* the pattern, the concrete skeleton to *execute* it, the complexity, and the single trap that *sinks* most attempts.

Table of Contents

- **Orientation:** [How to use this](#) · Pattern Selection · [Tier Mindsets](#) · Cross-Cutting Disambiguation
- **Tier 1 — Foundation:** [bfs](#) · [fast_slow_pointers](#) · [hashmap](#) · [merge_intervals](#) · [sliding_window](#) · [string](#) · [two_pointers](#)
- **Tier 2 — Intermediate:** [binary_search](#) · [bit_manipulation](#) · [design](#) · [dfs](#) · [divide_and_conquer](#) · [dynamic_programming](#) · [math](#) · [prefix_sum](#) · [randomized](#) · [stack](#) · [top_k_elements](#) · [two_heaps](#)
- **Tier 3 — Advanced:** [backtracking](#) · [cyclic_sort](#) · [modified_binary_search](#) · [subsets](#) · [trie](#)
- **Tier 4 — Expert:** [graph](#) · [greedy](#) · [matrix_traversal](#) · [monotonic_stack](#) · [union_find](#)

How to use this

- **Stuck on a new problem?** → Scan the **Pattern Selection** table; match the problem's signals to a pattern.
- **Learning a topic?** → Read its Mindset → When → Skeleton → Gotcha, then drill every problem in its table.
- **Tiers build on each other:** T1 (pointers / windows / hash) → T2 (heaps / DP / binary search) → T3 (combinatorial search) → T4 (advanced structures & invariants). Don't skip ahead — Tier 2+ problems are usually a Tier 1 pattern wrapped in extra state.

Pattern Selection (signal → pattern)

Signal in the problem statement	Use this pattern
sorted array + pair/triplet summing to target	two_pointers
longest / shortest / count of	sliding_window
subarrays/substrings under a constraint	
[start,end] ranges, meetings, coverage, timeline	merge_intervals
shortest path / min steps on an unweighted grid or graph	bfs
linked list cycle / middle / kth-from-end, O(1) space	fast_slow_pointers
count / frequency / membership / O(1) lookup / dedupe	hashmap
reformat / parse / case-validate strings	string
sorted / rotated / monotonic array, or O(log n) required	binary_search / modified_binary_search

Signal in the problem statement	Use this pattern
“minimize the max” / “maximize the min” + a feasibility test	<code>modified_binary_search</code> (answer-space)
return all combinations / permutations / subsets	<code>subsets</code> / <code>backtracking</code>
find a valid arrangement satisfying a constraint (prune hard)	<code>backtracking</code>
kth largest/smallest / top-k / k-closest	<code>top_k_elements</code>
median of a stream / sliding window	<code>two_heaps</code>
matching / nesting / decode (brackets, run-length)	<code>stack</code>
“next greater/smaller”, largest rectangle, subarray contribution	<code>monotonic_stack</code>
max/min ways , optimal substructure, overlapping subproblems	<code>dynamic_programming</code>
subarray sum/product = k , “equal # of X and Y”, range queries	<code>prefix_sum</code>
dependencies / prerequisites / valid ordering	<code>graph</code> (topological sort)
undirected “are these connected?” / redundant edge / same-set	<code>union_find</code>
prefix matching / autocomplete / word dictionary / wildcard	<code>trie</code>
numbers in range [1,n] (or [0,n]), find missing/duplicate, $O(1)$ space	<code>cyclic_sort</code>
“every element appears twice except one”, parity, bitmask	<code>bit_manipulation</code>
design a class with $O(1)$ ops / eviction policy	<code>design</code>
implement randX from randY , uniform sampling	<code>randomized</code>
matrix rotate/spiral/diagonal, or “nearest X per cell”	<code>matrix_traversal</code>
reachability / can-finish / “find the special node”	<code>graph</code> (degree arithmetic)
split → recurse → merge; counting inversions	<code>divide_and_conquer</code>

Tier Mindsets (the 30-second orientation)

- **Tier 1 — Foundation:** *Two cursors over a linear structure.* Pointers slide, windows expand/contract, hashes give $O(1)$ recall. ~70% of interview questions are a Tier 1 pattern in disguise. **Goal:** make pointer movement and hash-map lookups muscle memory.
- **Tier 2 — Intermediate:** *Add structure & state.* Heaps impose ordering, DP memoizes state, binary search halves a monotone space, trees/graphs need recursion + a visited set, design layers $O(1)$ structures. **Goal:** know *when* to invoke each — the hard part is recognition, not the mechanics.
- **Tier 3 — Advanced:** *Search a combinatorial space smartly.* Backtrack with pruning, enumerate with subsets, walk a trie, use the array as its own hash (cyclic sort), binary-search the *answer* not the array. **Goal:** disciplined choose→explore→un-choose, and recognizing feasibility-monotonic search.
- **Tier 4 — Expert:** *Advanced structures & invariants.* Union-find for dynamic connectivity, monotonic stacks for bounded-extremum problems, greedy proofs, topological order, matrix index-gymnastics. **Goal:** know the right invariant and when a greedy choice is provably safe vs when you must fall back to DP/multi-pass.

Tier 1 — Foundation

bfs

- **The Core Intuition (The “Aha!” Moment):** Imagine dropping a stone in a pond. Water rings move outward, step by step. BFS works exactly like this. It checks everything 1 step away, then 2 steps away, and so on. Because it searches level by level, the first time you find the target, it is guaranteed to be the shortest path.
- **Signal Recognition (When to use it):** Look for keywords like “shortest path”, “minimum steps”, “minimum minutes to spread”, “level by level traversal”, “nearest/closest”, or anything resembling multi-source propagation/radiation on a grid.
- **The Logical Blueprint (The Universal Steps):**
 1. Initialize a queue and push the starting node(s) (along with distance/level if needed).
 2. Mark the starting node(s) as visited immediately upon enqueueing.
 3. While the queue is not empty, process the current node (or an entire wave of nodes at once).
 4. For each unvisited neighbor, mark it as visited and enqueue it.
 5. Repeat until the target is found or the queue is exhausted.

- **The Minimal Skeleton:**

```
from collections import deque

def bfs(start, target):
    queue = deque([start])
    visited = {start} # Mark visited on ENQUEUE

    steps = 0
    while queue:
        # Process wave by wave
        for _ in range(len(queue)):
            node = queue.popleft()

            if node == target:
                return steps

            for neighbor in get_neighbors(node):
                if neighbor not in visited:
                    visited.add(neighbor) # Mark visited BEFORE enqueue
                    queue.append(neighbor)

        steps += 1

    return -1
```

- **Killer Gotchas:**
 - Mark visited **on enqueue**, never on dequeue! If you mark on dequeue, multiple nodes might enqueue the same neighbor, causing an exponential explosion (TLE).
 - In grid problems, you can often mutate the cell value directly (e.g., `grid[r][c] = VISITED`) to avoid $O(N)$ space for a visited set.

- When tracking steps, be careful not to increment for an empty wave at the end (e.g., in Rotting Oranges, only increment if `queue:` after each wave to avoid an off-by-one error).

• **Problem Table:**

Problem	Essence	Key Trick
p102 Level Order	Tree BFS → list-of-lists	<code>level_size = len(queue)</code> snapshots each row boundary
p449 Serialize/Deserialize BST	Two valid approaches	BFS Codec: level-order with "null" markers for both encode/decode. DFS Codec (compact): pre-order serialize (no nulls) + BST-bounds recursive rebuild (<code>build(lo, hi)</code>)
p463 Island Perimeter	BFS or $O(1)$ math	BFS: per cell, each OOB/water neighbor → +1. $O(1)$: <code>land*4 - 2*shared_edges</code> (scan right+down pairs only)
p513 Bottom Left Tree Value	Left-to-right BFS	Capture <code>queue[0].val</code> at the start of each level; last captured value is bottom-left (no reversal needed)
p515 Largest Value Each Row	Level-order, track max	<code>max_val = float("-inf")</code> , updated per node in level loop
p994 Rotting Oranges	Multi-source BFS	Seed all 2s at <code>t=0</code> ; count fresh upfront; if <code>queue: minutes += 1</code> after each wave; return <code>-1</code> if <code>fresh > 0</code>
p1091 Shortest Path Binary Matrix	Single-source BFS, 8 dirs	Mark <code>grid[r][c] = 1</code> on enqueue; early return when <code>neighbor == (n-1,n-1); n==1</code> → return 1 ; blocked start/end → <code>-1</code>

fast_slow_pointers

- **The Core Intuition (The “Aha!” Moment):** Imagine two cars driving on a road. One car is driving twice as fast as the other. If the road is straight, the fast car reaches the end exactly when the slow car is at the middle. If the road goes in a circle, the fast car will eventually catch up and hit the slow car from behind. This trick helps you find the middle or find a circle (cycle) using no extra memory.
- **Signal Recognition (When to use it):** You need to find the middle of a linked list, detect a cycle, or track a sequence defined by a mathematical transformation (like Happy Number) where $O(1)$ space is mandated or desired.
- **The Logical Blueprint (The Universal Steps):**
 1. Initialize two pointers, `slow` and `fast`.
 2. Advance `slow` by 1 step, and `fast` by 2 steps.
 3. Check the exit condition:
 - For cycle detection: Does `slow` equal `fast`?

- For middle finding: Has **fast** reached the end (or is about to)?

- **The Minimal Skeleton:**

```
# Finding cycle / middle of a linked list
slow = head
fast = head

while fast and fast.next: # Guard fast.next.next
    slow = slow.next
    fast = fast.next.next

    if slow is fast:
        return True # Cycle detected

return False # No cycle; slow is now at the middle
```

- **Killer Gotchas:**

- Always guard the fast pointer’s two-step jump: `while fast and fast.next:`. Trying to call `fast.next.next` when `fast.next` is `None` will crash with an `AttributeError`.
- When simulating on an array (index-based), guard with bounds checking: `while fast < len(arr) and fast + 1 < len(arr):`.
- For abstract sequences (like Happy Number), **offset the start**: `slow = n, fast = get_next(n)`. If both start at `n`, the `slow == fast` check will immediately terminate the loop on the first iteration.
- For even-length linked lists, the skeleton above lands `slow` on the **second** middle node. If you need the first middle node, stop when `fast.next.next` is `None` instead.

- **Problem Table:**

Problem	Difficulty	Key Trick	Essence
p141 Linked List Cycle	Easy	Both start at <code>head</code> , move then compare	Floyd’s: meet $\rightarrow$ cycle; fast hits <code>None</code> $\rightarrow$ false
p202 Happy Number	Easy	<code>fast = get_next(n)</code> (offset init); <code>get_next</code> abstracts the step	Chase sequence to 1 or cycle; return <code>fast == 1</code>
p876 Middle of Linked List	Easy	<code>fast+1 < len</code> guards 2-step; <code>slow</code> is result	Fast $2\times$ / slow $1\times$; slow lands on 2nd middle for even-length

hashmap

- **The Core Intuition (The “Aha!” Moment):** Imagine a giant library without a catalog. Finding a book takes a long time because you have to check every shelf. A hashmap is like a perfect catalog or an address book. You give it a name (the key), and it instantly tells you the location or information (the value). This turns a slow search into instant $O(1)$ magic.
- **Signal Recognition (When to use it):** Look for problems asking to count frequencies, group items, find pairs (like Two Sum), deduplicate, or implement $O(1)$ custom data structures (insert/remove/getRandom). Also useful for encoding/decoding bijectively.
- **The Logical Blueprint (The Universal Steps):**

1. Initialize a hashmap (or sets/arrays if keys are heavily constrained).
2. Iterate through the data, building the map (e.g., mapping value to index, or value to frequency).
3. Look back into the map as you iterate (or in a second pass) to resolve pairs, answer queries, or maintain invariants.

- **The Minimal Skeleton:**

```
# O(1) Insert/Remove/GetRandom pattern
class O1Set:
    def __init__(self):
        self.vals = []           # For O(1) random choice
        self.idx_map = {}       # val -> index in vals

    def insert(self, val):
        if val in self.idx_map: return False
        self.idx_map[val] = len(self.vals)
        self.vals.append(val)
        return True

    def remove(self, val):
        if val not in self.idx_map: return False

        # Swap with last element to enable O(1) pop
        idx = self.idx_map[val]
        last_val = self.vals[-1]

        self.vals[idx] = last_val
        self.idx_map[last_val] = idx # Update last_val's index FIRST

        self.vals.pop()
        del self.idx_map[val]        # Delete AFTER updating
        return True
```

- **Killer Gotchas:**

- In O(1) swap-and-pop (like above), you must update `idx_map[last_val] = idx` BEFORE you `del idx_map[val]`. If `val` is already the last element (self-swap), deleting first and then updating would leave a dangling reference.
- Reservoir sampling vs Hashmap: Use Reservoir Sampling (O(1) space) when stream length is unknown or doesn't fit in memory; otherwise, precomputing a dictionary of indices is faster for queries.
- When storing geometric coordinates or combinations, remember that tuples are hashable in Python, but lists are not.

- **Problem Table:**

Problem	Difficulty	Essence	Key Trick
p380 InsertDeleteGetRandom O(1)	Medium	All ops O(1) on a set	list + val→idx; remove = swap-with-last then pop+del
p382 Linked List Random Node	Medium	Uniform random from unknown-length stream	Reservoir: keep node i with prob 1/i via <code>randint(1,i)==1</code>

Problem	Difficulty	Essence	Key Trick
p391 Perfect Rectangle	Hard	Do sub-rects tile exactly?	XOR-set corners → only 4 outer survive; AND area sum check
p398 Random Pick Index	Medium	Pick uniform random index of target	Precompute <code>defaultdict(list)</code> of indices; <code>random.choice</code>
p447 Number of Boomerangs	Medium	Ordered equidistant triples per pivot	Sort dist list per pivot, run-length count → <code>c*(c-1)</code>
p500 Keyboard Row	Easy	Words typeable on one keyboard row	<code>set(word.lower())</code> row using 3 pre-built row sets
p535 Encode/Decode TinyURL	Medium	Bijjective URL shortener	Two dicts (url code) + base-62 auto-increment counter
p575 Distribute Candies	Easy	Max unique candy types $n/2$	<code>min(len(set(candyType)), n//2)</code> — unique count vs quota

merge_intervals

- **The Core Intuition (The “Aha!” Moment):** Imagine trying to clean up a messy calendar with many overlapping meetings. If the meetings are random, it is very confusing. But if you sort the meetings by their start time first, it becomes easy. You just look at the meetings one by one from left to right. If a meeting starts before the last one ends, you just combine them into one long meeting.
- **Signal Recognition (When to use it):** The input is an array of `[start, end]` intervals. The problem asks you to merge overlaps, insert a new interval, count meeting rooms, or find total covered time.
- **The Logical Blueprint (The Universal Steps):**
 1. Sort the intervals by their start times (if not already sorted).
 2. Initialize a list with the first interval.
 3. Iterate through the rest:
 - If the current interval overlaps with the last interval in the list, extend the last interval’s end time.
 - If it doesn’t overlap, append it as a new distinct interval.
- **The Minimal Skeleton:**

```
def merge_intervals(intervals):
    if not intervals: return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0][:]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]
```

```

if start <= last_end: # Overlap found
    # Extend the end (must take max to handle nested intervals)
    merged[-1][1] = max(last_end, end)
else:
    merged.append([start, end])

```

`return merged`

- **Killer Gotchas:**

- **Nested intervals:** Always use `max(last_end, end)` when merging. Don't just replace `last_end` with `end`, because the new interval might be completely swallowed by the last one (e.g., merging `[1, 10]` and `[2, 3]`).
- **Touching vs Overlapping:** Pay attention to the problem description. Usually, `[1, 4]` and `[4, 5]` overlap (use `<=`), but in meeting rooms, one meeting ending at 4 and another starting at 4 means the room can be recycled (use `<`).
- When finding peak concurrency (Meeting Rooms II), split the starts and ends into two independent sorted arrays. Treat it as a chronology of events (starts = +1 room, ends = -1 room), sweeping through and recycling rooms when `start >= ends[end_ptr]`.

- **Problem Table:**

Problem	Difficulty	Essence	Key trick
p056 Merge Intervals	Medium	Sort by start; extend <code>last.end</code> on overlap	<code>max(last.end, end)</code> handles nesting; <code><=</code> catches touching
p057 Insert Interval	Medium	3-phase linear scan (before / overlap-merge / after)	Phase boundary: strict <code><</code> then inclusive <code><=</code> ; $O(N)$, no sort
p253 Meeting Rooms II	Medium	Split into sorted starts+ends; count starts without a free room	<code>start >= ends[ptr]</code> (, not <code>></code>) recycles room; touching = free
p495 Teemo Attacking	Easy	Sum <code>min(duration, gap)</code> for each consecutive pair	Always <code>+duration</code> at end for last attack's full window

sliding_window

- **The Core Intuition (The “Aha!” Moment):** Imagine a box that slides over a row of items. When the box moves to the right, you do not need to look at every item inside the box again. You only add the new item entering the box, and remove the old item leaving the box. This saves a lot of time. It changes slow code ($O(N^2)$) into fast code ($O(N)$).
- **Signal Recognition (When to use it):** Look for keywords like “contiguous subarray”, “longest/shortest substring”, “maximum sum of size K”, “anagrams/permutations within a string”. If the problem asks for optimal consecutive elements and the array/string has positive values or a frequency constraint, sliding window is your go-to.
- **The Logical Blueprint (The Universal Steps):**

1. Initialize `left` and `right` pointers at the start of the array/string, and a state tracker (e.g., sum, hash map).

2. Expand the window by moving **right** and adding the new element to your state.
3. Check if the window is invalid (e.g., sum too large, too many distinct characters). If invalid, continuously shrink the window by removing the **left** element from your state and moving **left** forward until it becomes valid again.
4. At every valid step, update your global answer (e.g., max length, min length, total count).
5. Repeat until **right** reaches the end.

- **The Minimal Skeleton:**

```
def sliding_window(nums, k):
    left = 0
    state = 0 # Can be a sum, a counter, etc.
    best = 0

    for right, val in enumerate(nums):
        # 1. Expand: add nums[right] to state
        state += val

        # 2. Shrink: while window is invalid
        while state > k: # Replace with actual invalid condition
            state -= nums[left]
            left += 1

        # 3. Update best answer
        best = max(best, right - left + 1)

    return best
```

- **Killer Gotchas:**

- **Stale Pointers/Data:** If jumping **left** directly (e.g., to skip duplicates using a dictionary of last seen indices), make sure you don't jump backward. Always **left = max(left, last_seen[char] + 1)**.
- **Updating the Answer:** Know *when* to update your answer. If you need the *longest* valid window, update it *after* the **while** loop makes the window valid. If you need the *shortest* valid window, update it *inside* the **while** loop *before* shrinking.
- **Zero Counts in Hash Maps:** When maintaining character frequencies with a Python Counter or dict, **state[char] == 0** is NOT the same as the character missing. Actively **del state[char]** if it hits 0 so equality comparisons (e.g., **window == target**) work perfectly.

- **Problem Table:** | Problem | Pattern | Essence / Key Trick | |—|—|—| | p003 Longest Substring No Repeat | Variable | **char→last_index**; jump **left = last_index+1** only if index **left** (stale guard) | | p395 Longest Substring K Repeating | Variable (×26) | Loop **t** in 1..26; window with exactly **t** unique chars, all freq **k**; **unique==t** and **at_least_k==t** | | p424 Longest Repeating Char Replacement | Variable | **freq + max_freq**; valid iff **window_len - max_freq ≤ k**; **max_freq** never decreases | | p438 Find All Anagrams | Fixed | Rolling Counter vs **p_count**; delete zero-count keys so **==** works correctly | | p567 Permutation in String | Fixed | **list[26]** arrays; roll by +1/-1; list equality avoids zero-key issue entirely |

string

- **The Core Intuition (The “Aha!” Moment):** Strings are just lists of characters. Most string problems are easy if you clean the data first. For example, remove spaces or make

everything lowercase before you start. You do not need complex tools. Just use simple flags (like `is_inside_word = True`) or built-in language tools (like `.split()`).

- **Signal Recognition (When to use it):** The input is strings and asks to reformat them (license keys, URLs), validate rules (capitalization, character classes), group characters, or count segments/words.
- **The Logical Blueprint (The Universal Steps):**
 1. **Normalize:** Strip noise, convert cases, or remove invalid characters first. It's infinitely easier to work with clean data than to handle edge cases mid-loop.
 2. **Scan or Rebuild:** Either use a boolean flag while scanning left-to-right (to track if you are inside a word/segment) OR chunk the normalized string using slices to rebuild it.
 3. **Return:** Re-join chunks if you're formatting, or return the boolean/counter.

- **The Minimal Skeleton:**

```
# Pattern A: State Machine (Scanning)
def count_segments(s: str) -> int:
    count, in_seg = 0, False
    for ch in s:
        if ch != ' ' and not in_seg:
            count += 1
            in_seg = True
        elif ch == ' ':
            in_seg = False
    return count

# Pattern B: Normalize and Rebuild
def reformat(s: str, k: int) -> str:
    cleaned = s.replace("-", "").upper()
    # Process the first irregular chunk
    first_len = len(cleaned) % k
    groups = [cleaned[:first_len]] if first_len else []

    # Process remaining perfectly sized chunks
    for i in range(first_len, len(cleaned), k):
        groups.append(cleaned[i:i+k])

    return "-".join(groups)
```

- **Killer Gotchas:**

- **First vs Last Chunk:** When splitting into groups of size `k`, the *first* group is often the short one (`len % k`), not the last one. Calculate `first_len` and slice from the front!
- **Built-in Blindness:** Don't write a 15-line loop to check if all letters are lowercase when `word.islower()` exists. Check `word[1:].islower()` to elegantly handle Title Case and single-character words simultaneously.
- **String Equality Illusion:** Equal length does not mean equal string. If two strings are entirely different, neither is a subsequence of the other, so the longest uncommon subsequence is simply `max(len(a), len(b))`.

- **Problem Table:** | Problem | Complexity | Key trick | |---|---| | p434 Number of Segments | $O(N)$ / $O(1)$ | `in_segment` flag: count space→non-space transitions; or just `len(s.split())` | | p482 License Key Formatting | $O(N)$ / $O(N)$ | Strip+upper first; **first** group = `len%k` (front slice), then `k`-chunks joined by `-` | | p520 Detect Capital | $O(N)$ / $O(1)$ | 1-liner: `isupper` or

`islower` or `word[1:].islower()` — `[1:]` covers single chars too | | p521 Longest Uncommon Subseq I | $O(N)$ / $O(1)$ | `a == b` (string equality!) $\rightarrow -1$; else `max(len(a), len(b))` — NOT `len(a)==len(b)` |

two_pointers

- **The Core Intuition (The “Aha!” Moment):** If a list is already sorted, you do not need to check every pair of numbers. You can put one finger at the start and one finger at the end. Then, move your fingers toward the middle based on what you need. If the sum is too small, move the left finger to the right to get a bigger number. If the sum is too big, move the right finger to the left. This makes your search very fast.
- **Signal Recognition (When to use it):** Look for “sorted array”, “find pairs/triplets that sum to X”, “container with most water”, “k-diff pairs”, or checking if one string is a “subsequence” of another.
- **The Logical Blueprint (The Universal Steps):**
 - **Inward Pointers (Pairs/Sums):** Place `left` at start, `right` at end. Calculate sum/area. If too small, move `left` right. If too big, move `right` left. If equal, record it and move *both*.
 - **Same-Direction Pointers (Deduplication/Subsequence):** Place `slow` and `fast` at start. `fast` scouts ahead to find valid conditions. `slow` marks where the next valid element should go, or acts as an anchor.
 - **Co-Advancing Sequences (Merging):** Pointer `i` on sequence A, pointer `j` on sequence B. Compare `A[i]` and `B[j]`, then selectively advance one or both.
- **The Minimal Skeleton:**

```
# Pattern: Inward Pointers (Two Sum on Sorted Array)
def two_sum_sorted(nums: list[int], target: int) -> list[int]:
    left, right = 0, len(nums) - 1

    while left < right:
        curr = nums[left] + nums[right]
        if curr == target:
            return [left, right]
        elif curr < target:
            left += 1 # Need a bigger sum
        else:
            right -= 1 # Need a smaller sum

    return []

# Pattern: Is Subsequence (Same Direction)
def is_subseq(s: str, t: str) -> bool:
    i = 0
    for ch in t:
        if i < len(s) and s[i] == ch:
            i += 1
    return i == len(s)
```

- **Killer Gotchas:**
 - **The Unsorted Trap:** Inward two-pointers for sums *require* the array to be sorted. If it’s unsorted and you sort it, remember that original indices are lost.

- **Infinite Loops on Duplicates:** When deduplicating pairs/triplets (like in 3Sum), you must use a `while` loop to skip over duplicates *after* you record a valid pair, and you must move *both* pointers.
- **Colliding Pointers:** In same-direction pointers (like finding differences), ensure `left != right`. If they collide, always forcefully advance `right += 1` to keep them separated.
- **1-Indexed Results:** Read carefully. Some two-pointer problems (like Two Sum II) want 1-indexed output, meaning you return `[left + 1, right + 1]`.
- **Problem Table:** | Problem | Difficulty | Essence | Key Trick | |---|---|---| | p011 Container With Most Water | Medium | Maximize `min(h[l], h[r]) * (r - l)` | Move the **shorter** side inward; taller side can't improve area | | p015 3Sum | Medium | All unique triplets summing to 0 | Sort + fix `i` + inward two-pointer; **3-level dedup** | | p167 Two Sum II | Medium | Pair sum in sorted array | Inward pointers; return **1-indexed** result | | p392 Is Subsequence | Easy | Is `s` embeddable in `t` by deletions? | `is_subseq` helper: advance `i` only on char match | | p475 Heaters | Medium | Min radius to cover all houses | Sort both; co-advance heater `j` while it gets closer; never reset `j` | | p522 Longest Uncommon Subseq II | Medium | Longest string not a subseq of any other | Sort by `len` desc; use `is_subseq`; duplicates always disqualify | | p524 Longest Word via Deleting | Medium | Longest dict word that's subseq of `s` | `is_subseq` per word; tie-break: longer > lexicographically smaller | | p532 K-Diff Pairs | Medium | Count unique pairs with `|diff| == k` | Sort + same-dir two-pointer; skip dups after match; guard `left == right` |

Tier 2 — Intermediate

binary_search

- **The Core Intuition (The “Aha!” Moment):** Think of guessing a number from 1 to 100. If you guess 50 and I say “lower”, you do not need to check 51 to 100. You just threw away half of the options! Binary search does exactly this. It uses a simple “yes or no” rule to cut the search area in half every time. It keeps cutting until it finds the exact answer.
- **Signal Recognition (When to use it):** “Sorted array”, “rotated array”, “find the first or last position”, “ $O(\log n)$ time complexity required”, “leftmost satisfying a condition” (like the first bad version), or “minimum/maximum possible value” (binary searching the answer).
- **The Logical Blueprint (The Universal Steps):**
 1. Define the start (`left`) and end (`right`) of your search area.
 2. Find the exact middle point (`mid`).
 3. Ask a “yes or no” question about the `mid` point.
 4. Depending on the answer, throw away the half where the target cannot be.
 5. Repeat this process until `left` and `right` meet and give you the answer.
- **The Minimal Skeleton:**

```
# Template 1: Exact Match (Find a specific target)
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2 # Find middle safely
    if nums[mid] == target:
        return mid # Found it
    elif nums[mid] < target:
        left = mid + 1 # Throw away left half
    else:
```

```
        right = mid - 1          # Throw away right half
    return -1                    # Not found

# Template 2: Find the boundary (Leftmost True)
left, right = 0, n
while left < right:             # Notice: < not <=
    mid = left + (right - left) // 2
    if is_good(mid):
        right = mid             # Keep mid, it might be the answer
    else:
        left = mid + 1          # Move past bad items
return left                     # left and right converge
```

- **Killer Gotchas:**

- **Infinite loop:** Mixing up the templates. `while left < right` must use `right = mid`. `while left <= right` must use `right = mid - 1`. Do not mix them.
- **Number Overflow:** `(left + right) // 2` can crash in languages like C++ or Java for very large numbers. Always use `left + (right - left) // 2`.
- **Rotated Arrays:** When finding the minimum in a rotated array, always compare `mid` with `right`. If you compare it with `left`, you will get wrong answers.

- **Problem Table:** | Problem | Difficulty | Essence | Key trick | |---|---|---| | p704 Binary Search | Easy | Classic exact-match | Template A; return `mid` on hit, `-1` after loop | | p153 Find Min Rotated | Medium | Boundary of rotation | `nums[mid]>nums[right]→left=mid+1`; else `right=mid`; return `nums[left]` | | p278 First Bad Version | Easy | Leftmost True predicate | Template B; `is_bad(mid)→right=mid`; else `left=mid+1` | | p352 Data Stream Disjoint Intervals | Hard | Sorted interval list + merge on insert | `bisect_left` on `[lo,hi]` pairs; merge left if `prev[1]>=lo-1`; merge right while `cur[0]<=hi+1` | | p354 Russian Doll Envelopes | Hard | 2-D LIS → 1-D LIS | Sort ($w\uparrow, h\downarrow$); `bisect_left` on `heights`; replace or append (patience sort) | | p483 Smallest Good Base | Hard | Geometric sum = n ? | Outer loop `m` from `bit_length(n)` down to 2; inner BS `k` in `[2, n^(1/(m-1))+2]`; fallback = `n-1` | | p497 Random Point in Rectangles | Medium | Weighted random rect | Prefix integer-point counts; `randint(1,total)`; Template B to find rect; `randint` within rect |

bit_manipulation

- **The Core Intuition (The “Aha!” Moment):** Computers store numbers as a line of 0s and 1s. Instead of using normal math like plus or minus, we can change these 0s and 1s directly. Think of it like a row of light switches. You can turn a switch on, turn it off, or flip it. This is super fast because it is exactly how the computer naturally works.
- **Signal Recognition (When to use it):** “Every element appears twice except one”, “find the missing number”, “do not use division”, “ $O(1)$ extra space”, “count the number of 1s”, or “check if it is a power of two”.
- **The Logical Blueprint (The Universal Steps):**
 1. Identify what you want to do with the bits (count them, find a unique one, flip them).
 2. Use XOR ($\wedge$) to cancel out matching pairs (because $X \text{ XOR } X = 0$).
 3. Use AND ($\&$) with `n - 1` to turn off the lowest “1” bit in a number.
 4. Use shifts (`>>` or `<<`) to check the bits one by one.
- **The Minimal Skeleton:**

```
# 1. XOR Fold: Find the single odd element out
res = 0
for x in nums:
    res ^= x          # Pairs cancel out to 0, only unique remains

# 2. Brian Kernighan: Count the "1" bits fast
count = 0
while n > 0:
    n &= (n - 1)       # Turns off the rightmost "1" bit
    count += 1         # Loops only for the number of "1"s

# 3. DP on bits: Build count for 0 to n quickly
ans = [0] * (n + 1)
for i in range(1, n + 1):
    ans[i] = ans[i >> 1] + (i & 1)  # Shift right and add the lost bit
```

- **Killer Gotchas:**

- **Python's ~ operator:** ~n in Python does not just flip the bits; it gives -(n+1) because numbers have infinite length. To flip bits, always create a mask (like 1111) and use XOR: n ^ mask.
- **n &= n - 1 vs Subtracting 1:** n &= n - 1 clears the lowest "1" bit. It is not the same as just subtracting 1. This makes counting bits super fast.
- **The Zero Trap:** Be careful with 0. The power-of-two check is n > 0 and (n & (n - 1)) == 0. If you forget n > 0, the number 0 will wrongly pass the test!

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p136 Single Number | XOR all — pairs cancel | res=0; for x in nums: res^=x — O(n)/O(1) | | p0191 Number of 1 Bits | Count set bits | Brian Kernighan: n &= n-1 loops exactly popcount(n) times | | p338 Counting Bits | Bit counts for 0..n | DP: ans[i] = ans[i>>1] + (i&1) — right-shift shares subproblem | | p476 Number Complement | Flip significant bits only | mask=(1<<num.bit_length())-1; return num^mask — avoids Python ~ trap |

design

- **The Core Intuition (The “Aha!” Moment):** Imagine you are running a very busy restaurant. If you have to look through a long list to find a customer's order, it takes too long. Instead, you keep multiple fast-access lists. One list connects a name to an order. Another list groups orders by time. By keeping these fast lists updated together, every action becomes instant. You use more paper (memory) to save time (speed).
- **Signal Recognition (When to use it):** “Design a class”, “implement a cache”, “O(1) average time complexity”, “LRU (Least Recently Used) or LFU (Least Frequently Used)”, or “eviction policy”.
- **The Logical Blueprint (The Universal Steps):**
 1. Use a Dictionary (Hash Map) to find values instantly using a key.
 2. Use a second data structure (like another Dictionary, a Set, or an OrderedDict) to track the rules, like frequency or recency.
 3. When you read or write data, update all data structures at the exact same time so they never disagree.
 4. If the cache is full, use your tracking structure to find the right item to remove (evict).

- **The Minimal Skeleton:**

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.cache = OrderedDict() # Remembers the order keys were added

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key) # Mark as recently used
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if self.cap <= 0:
            return
        if key in self.cache:
            self.cache.move_to_end(key) # Mark as recently used
        self.cache[key] = value

        if len(self.cache) > self.cap:
            # Pop the oldest item (from the beginning)
            self.cache.popitem(last=False)
```

- **Killer Gotchas:**

- **Empty Buckets:** In LFU caches, always delete empty frequency buckets. If you leave them empty, your “minimum frequency” tracker will look at the empty bucket and evict nothing, causing a crash.
- **Updating vs Inserting:** If you `put()` a key that already exists, it is an update, not a new insert. Do not increase the total size, and do not evict anything.
- **Zero Capacity:** Always check if `capacity <= 0` at the very beginning of your `put` function. LeetCode loves to test with a cache size of 0 to see if your code breaks.

- **Problem Table:** | Problem | Difficulty | Essence | Key trick | |—|—|—|—| | p460 LFU Cache | Hard | 3-dict LFU: key→val, key→freq, freq→bucket | Delete empty buckets; track `min_freq`; `OrderedDict` per bucket for true $O(1)$ LRU tie-break |

dfs

- **The Core Intuition (The “Aha!” Moment):** Think of walking through a maze. You keep walking down one path until you hit a dead end. When you hit a dead end, you take one step back and try the next possible path. You keep a map and mark where you have been so you never walk in a circle. This is Depth-First Search. It goes as deep as possible first before looking at other options.
- **Signal Recognition (When to use it):** “Find all possible ways”, “number of islands”, “connected components”, “walk through a tree”, “explore a grid”, or “valid paths from start to finish”.
- **The Logical Blueprint (The Universal Steps):**

1. Write a function that takes your current position (like a node, or row and column).

2. Check if you are out of bounds or at a bad position. If yes, stop and go back.
3. Mark the current position as “visited” so you do not come back to it.
4. Look at all valid neighbors (up, down, left, right, or children in a tree).
5. Call the exact same function (recursion) for each neighbor to explore deeper.

- **The Minimal Skeleton:**

```
def count_components(grid):
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        # 1. Stop if out of bounds or not the target
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != "1":
            return

        # 2. Mark as visited BEFORE moving
        grid[r][c] = "0"

        # 3. Explore all 4 directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                dfs(r, c)

    return count
```

- **Killer Gotchas:**

- **Mark BEFORE you move:** Always mark a node as ‘visited’ before you call DFS on its neighbors. If you forget or do it after, your program will run forever in a loop between two nodes.
- **Python recursion limits:** In Python, recursion can go too deep and crash the program. For very large grids, you might need to use an explicit stack (a list) instead of recursion.
- **Grid Boundaries:** Always check `r < 0` or `r >= rows` before you read `grid[r][c]`. Reading outside the grid will cause an “index out of range” error.

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p200 Number of Islands | Count connected '1' components | Sink cells ('1'→'0') in-place during DFS; no visited set needed | | p695 Max Area of Island | Largest connected 1 component | `dfs` returns `1 + sum(4 neighbors)`; sink to 0 before recursing | | p572 Subtree of Another Tree | Does `subRoot` appear as an exact subtree? | Two-level DFS: outer `is_subtree` walks all nodes; inner `is_same` checks structural equality | | p508 Most Freq Subtree Sum | Post-order sum → frequency map | `subtree_sum` returns int and side-effects `freq` dict; filter by `max(freq.values())` | | p538 Convert BST → Greater Tree | Each node gets += sum of all larger nodes | Reverse in-order (R→N→L) with `nonlocal total` accumulator; modifies in-place | | p450 Delete Node in BST | BST delete handling 3 cases | 2-child case: copy inorder successor value up, then recursively delete successor from right subtree | | p1306 Jump Game III | Can index with value 0 be reached via ±jumps? | DFS on index graph; `visited.add(i)` before branching to avoid cycles | | p565 Array Nesting

| Longest cycle in permutation array | Iterative cycle-follow with boolean visited array; mark before advancing `j = nums[j]` |

divide_and_conquer

- **The Core Intuition (The “Aha!” Moment):** Imagine you have 100 unsorted cards. Sorting them all at once is hard. Instead, you split the cards into two piles of 50. You give each pile to a friend to sort. They keep splitting until everyone has just 1 card. One card is already sorted! Then, you take the sorted small piles and easily merge them together. The secret of Divide and Conquer is in the “merge” step. Splitting is easy, but putting the solved pieces back together is where the real work happens.
- **Signal Recognition (When to use it):** Look for problems that ask to “sort an array”, “count inversions/reverse pairs”, or “construct a quad-tree”. Phrases like “merge k sorted...” or recurrence relations resembling $T(n) = 2T(n/2) + O(n)$ are dead giveaways. Also, if a problem is naturally represented as a 2D grid that needs to be broken into 4 quadrants, it’s a D&C problem.
- **The Logical Blueprint (The Universal Steps):**
 1. **Base Case:** Define the simplest possible input (e.g., an array of length 0 or 1) and return the immediate answer.
 2. **Divide:** Split the problem space (array, grid, etc.) into equal or roughly equal halves (or quadrants).
 3. **Conquer:** Recursively call the same function on these divided parts.
 4. **Combine:** This is the core logic. Take the results from the recursive calls and merge them, count across them, or link them together to form the answer for the current level.
- **The Minimal Skeleton:**

```
def divide_and_conquer(arr):
    # 1. Base case
    if len(arr) <= 1:
        return arr[:] # Return a copy to avoid mutating the original prematurely

    # 2. Divide
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    # 3. Conquer
    left_res = divide_and_conquer(left_half)
    right_res = divide_and_conquer(right_half)

    # 4. Combine (Example: Merge step for sorting/counting)
    result = []
    i = j = 0
    while i < len(left_res) and j < len(right_res):
        if left_res[i] <= right_res[j]:
            result.append(left_res[i])
            i += 1
        else:
            result.append(right_res[j])
            j += 1

    result.extend(left_res[i:])
```

```
result.extend(right_res[j:])  
  
return result
```

- **Killer Gotchas:**

- **Slicing doubles memory:** Using `arr[:mid]` creates a copy at every recursive level, leading to $O(n \log n)$ total extra space. If memory is tight, pass indices (`start`, `end`) instead of slicing.
- **Phase ordering in counting pairs (e.g., Reverse Pairs):** You must COUNT cross pairs using the original values *before* you SORT the subarray. Swapping these phases destroys the relative pair relationships and gives wrong answers.
- **Cross-pair condition:** When counting pairs like `nums[i] > 2 * nums[j]`, remember this is a specific multiplier condition, not a standard inversion.
- **Quad-Tree scanning costs:** Scanning an entire grid to check if it's uniform is $O(\text{size}^2)$. In the worst case, this leads to $O(n^2 \log n)$ time complexity, not $O(n \log n)$.

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p912 Sort an Array | Classic merge sort | Split at `mid`, recurse, two-pointer merge; base `len 1` returns a copy | | p023 Merge k Sorted Lists | D&C on the list-of-lists array | Split lists array in half, recurse each half to a flat sorted array, two-pointer merge | | p169 Majority Element | Boyer-Moore voting (NOT actual D&C) | `count==0` → new candidate; works only when majority > $n/2$ is guaranteed | | p427 Construct Quad Tree | Recursive 2-D subdivision | `all_same(x,y,size)` → leaf; else split into 4 quadrants; BFS serialize with nulls | | p493 Reverse Pairs | Merge-sort counting with $2\times$ condition | Count `left[i] > 2*right[j]` cross-pairs (two-pointer) BEFORE sorting the subarray |

dynamic_programming

- **The Core Intuition (The “Aha!” Moment):** Dynamic Programming is “smart brute force.” Imagine calculating the 10th Fibonacci number. Instead of recalculating the 3rd and 4th numbers many times, you calculate them once, save the answers in a list, and look them up later. Every DP problem asks you to define a “state” (what does one item in the list mean?) and a “transition” (how do I use old answers to find the new answer?).
- **Signal Recognition (When to use it):** The prompt asks for the “maximum/minimum cost”, “number of distinct ways”, or “is it possible to...”. The problem involves making a sequence of choices where current choices depend on previous ones (overlapping subproblems), and the optimal solution can be constructed from optimal sub-solutions (optimal substructure). Often applied to sequences, strings (substrings/subsequences), or grids.
- **The Logical Blueprint (The Universal Steps):**
 1. **Define the State:** Figure out what changing variables uniquely identify a subproblem (e.g., `dp[i]` = max profit up to day `i`).
 2. **Find the Base Case(s):** Determine the trivial starting values (e.g., `dp[0]` = 0 or `dp[1]` = `nums[0]`).
 3. **Establish the Recurrence Relation:** Formulate how to compute the current state from previously computed states (e.g., `dp[i]` = `max(dp[i-1], dp[i-2] + nums[i])`).
 4. **Choose Iteration Order:** Decide if you are building from bottom-up (loops) or top-down (recursion + memoization).
 5. **Extract the Answer:** The final result is usually at the end of your DP table (`dp[n]`) or the max/min of the entire table.
- **The Minimal Skeleton:**

```
# Skeleton: 1D Bottom-Up DP (e.g., House Robber / Fibonacci)
def solve_1d_dp(nums):
    if not nums: return 0
    n = len(nums)

    # 1. Define State and Base Cases
    dp = [0] * (n + 1)
    dp[0] = 0 # Base case 1
    dp[1] = nums[0] # Base case 2

    # 2. Iterate and Transition
    for i in range(2, n + 1):
        # Choose to either skip current, or take current + non-adjacent previous
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])

    # 3. Extract Answer
    return dp[n]

# Skeleton: Unbounded Knapsack (Combinations / Min Items)
def solve_knapsack(items, target):
    dp = [0] * (target + 1) # Use float('inf') for minimums
    dp[0] = 1 # Base case: 1 way to make target 0

    for item in items: # Outer loop over items prevents permutation duplicates
        for amount in range(item, target + 1):
            dp[amount] += dp[amount - item]

    return dp[target]
```

- **Killer Gotchas:**

- **Combinations vs. Permutations in Knapsack:** When counting ways, if the outer loop is over `items` and the inner loop is over `amounts`, you get *combinations* (order doesn't matter). If you swap them (outer `amounts`, inner `items`), you get *permutations* (1+2 and 2+1 are counted twice).
- **INF Sentinel for Min-DP:** When looking for a minimum (like fewest coins), initialize the DP table with `amount + 1` or `float('inf')`. Always check at the end if the answer is still INF (meaning impossible) and return -1 if so.
- **Interval DP Loop Order:** For palindromes or bursting balloons, you cannot iterate `i` from 0 to `n`. You MUST iterate by `length` of the subarray first, so that smaller internal intervals (`dp[i+1][j-1]`) are solved before expanding to (`dp[i][j]`).
- **Inverted Framing (Egg Drop):** Sometimes the naive state `dp[eggs][floors]` is too slow. Invert the question: `dp[moves][eggs] = max floors testable`.

- **Problem Table:** | Problem | Pattern | Essence | Key Trick | |---|---|---| | p070 Climbing Stairs | 1D linear | `dp[i]=dp[i-1]+dp[i-2]` | Two rolling vars; this is Fibonacci | | p198 House Robber | 1D linear | `curr = max(curr, prev + num)` | Rob-or-skip; no array needed | | p322 Coin Change | Unbounded knapsack (min) | `dp[i] = min(dp[i-coin]+1)` | Init with `amount+1`; return -1 if still INF | | p518 Coin Change II | Unbounded knapsack (count) | `dp[a] += dp[a-coin]` | Outer loop = **coins** → combinations not permutations | | p516 Longest Palindromic Subseq | Interval DP | Match ends +2; else max of shrink one side | Iterate by length; base `dp[i][i]=1` | | p403 Frog Jump | Top-down memo | `can_reach(pos, k)` → try `k-1/k/k+1` | Use `set(stones)` for `O(1)` lookup; prune `step > 0` | | p887 Super Egg Drop | Inverted DP | `dp[m][k] = dp[m-1][k-1]+dp[m-1][k]+1` | Reframe: “max floors with `m` moves”; find min `m`

n | | p514 Freedom Trail | 1D backward DP | $dp[pos] = \min(\text{rotate_cost} + dp[\text{target}])$ | Circular cost $\min(\text{diff}, n - \text{diff}) + 1$; iterate key backward | | p552 Student Attendance II | 3D state-machine | $dp[i][\text{absences}][\text{consec_lates}]$ | 3 transitions: P/A/L; mod 10 + 7 | | p576 Out of Boundary Paths | 3D grid DP | $dp[\text{move}][r][c]$ paths remaining | Count out-of-bounds moves as exits; mod 10 + 7 | | p467 Unique Wrap Substrings | Char-endpoint DP | $\text{max_len}[c] = \text{max run length ending at } c$ | Wraparound: $\text{diff} == 1$ or $\text{diff} == -25$; $\text{answer} = \text{sum}$ | | p466 Count Repetitions | Cycle detection | Simulate s2 match through s1 copies; detect s2_index cycle | Skip full cycles; check charset subset first | | p553 Optimal Division | Greedy (not DP!) | Always $a/(b/c/\dots/z) = a*c*\dots*z/b$ | Math insight: maximise divisor = group all after first under one paren |

math

- **The Core Intuition (The “Aha!” Moment):** Math problems in coding interviews ask you to find a hidden formula. It is like counting bricks in a wall: you do not count one by one, you just multiply rows by columns. You should not use `while` loops that run millions of times. Instead, find a mathematical rule, use a simple formula, or generate a few correct choices directly.
- **Signal Recognition (When to use it):** Look for problems that ask for the “minimum number of tests/pigs” (screams information theory), “random point in a circle” (geometry/sampling), “next permutation” of a number, or finding a “closest palindrome”. Extremely large input constraints (like numbers up to 10^{18}) combined with strict time limits are a strong signal that an $O(1)$ math formula or an $O(\log n)$ digit manipulation is required.
- **The Logical Blueprint (The Universal Steps):**
 1. **Identify the Core Principle:** Is this information theory (base conversion)? Geometry (rejection sampling)? Combinatorics (permutations)?
 2. **Avoid Simulation:** If you find yourself wanting to increment a number by 1 inside a `while` loop that could run a billion times, stop. There is a mathematical shortcut.
 3. **Generate Candidates / Apply Formula:** For geometry or information theory, apply the mathematical formula. For strings/numbers (like palindromes or permutations), generate the small set of valid candidates (e.g., mirroring prefixes) or apply the specific permutation algorithm.
 4. **Handle Edge Cases:** Math formulas notoriously break on extreme bounds (like $n=1$, single digits, or overflow limits).
- **The Minimal Skeleton:**

```
# Skeleton: Next Greater Element / Next Permutation
def next_permutation_number(n):
    digits = list(str(n))
    length = len(digits)

    # 1. Find the first dip from the right
    i = length - 2
    while i >= 0 and digits[i] >= digits[i + 1]:
        i -= 1

    if i < 0:
        return -1 # No greater permutation possible

    # 2. Find the smallest digit to the right that is strictly greater than digits[i]
    j = length - 1
    while digits[j] <= digits[i]:
```

```
j -= 1

# 3. Swap and reverse the suffix
digits[i], digits[j] = digits[j], digits[i]
digits[i + 1:] = reversed(digits[i + 1:])

result = int("".join(digits))

# 4. Check integer bounds!
return result if result <= 2**31 - 1 else -1
```

- **Killer Gotchas:**

- **Rejection Sampling (Geometry):** When picking a random point in a circle, do NOT pick a random radius uniformly. This clusters points near the center because area grows with r^2 . Instead, pick a random point in a square bounding box and reject it if it falls outside the circle.
 - **Information Theory States (+1):** In the “Poor Pigs” problem, the base of the exponent is `tests + 1`, not `tests`. The +1 represents the implicit state of “survived all rounds.”
 - **Tie-Breaking Closest Numbers:** When looking for closest candidates (like palindromes), if two candidates have the same absolute difference from the target, you usually must return the *smaller* one. Use Python’s `min()` with a tuple key: `key=lambda x: (abs(x - target), x)`.
 - **Overflow Constraints:** Python automatically handles arbitrarily large integers, but LeetCode problems often demand a 32-bit signed integer limit. You must explicitly verify `result <= 2**31 - 1`.
 - **Edge Case Breakdowns:** Generic math algorithms (like prefix mirroring for palindromes) often fail for 1-digit numbers or exact powers of 10. You must manually add candidates like $10^{len-1} - 1$ (e.g., 999) or $10^{len} + 1$ (e.g., 1001).
- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p458 Poor Pigs | Min pigs s.t. $(tests+1)^{pigs}$ buckets | Information theory: each pig = 1 digit in base `tests+1`; loop until capacity buckets | | p478 Random Point in Circle | Uniform random point inside circle of radius r | Rejection sample in unit square $[-1, 1]^2$; $\sim 78.5\%$ acceptance; then scale by r | | p479 Largest Palindrome Product | Largest palindrome = product of two n -digit numbers; return mod 1337 | Mirror top-half $\rightarrow$ candidate palindrome; scan right from upper until `right2 < pal`; $n=1 \rightarrow 9$ | | p556 Next Greater Element III | Next permutation of digits; -1 if none or $> 2^{31}-1$ | Standard next-permutation: find dip from right, swap with next-larger, reverse suffix | | p564 Find Closest Palindrome | Nearest palindrome (not itself); tie $\rightarrow$ smaller | 5 candidates: `prefix $\pm$ 1` mirrored, same prefix mirrored, $10^{len-1}-1$, $10^{len}+1$ |

prefix_sum

- **The Core Intuition (The “Aha!” Moment):** A prefix sum is like a running bank balance. If your total balance in January was \$100, and in March it was \$500, you know you made \$400 between February and March. You do not need to look at every transaction. By saving a running total, you can find the sum of any subarray instantly using subtraction: `Sum(i to j) = Prefix[j] - Prefix[i-1]`.
- **Signal Recognition (When to use it):** Look for phrases like “subarray sum equals K ”, “contiguous array with equal 0s and 1s”, “subarray product”, or questions asking for properties of a contiguous range. If division is explicitly banned (e.g., “Product of Array Except Self”), two-pass prefix/suffix arrays are the intended solution. Also applies to weighted random sampling.
- **The Logical Blueprint (The Universal Steps):**

1. **Initialize the Accumulator and Map:** Create a running total (`curr_sum` or `balance`) and a Hash Map to store past prefix states.
2. **Set the Base Case:** *Crucial step.* Prime the map to account for a subarray that starts from the very beginning (index 0). For sums, `map[0] = 1` (frequency) or `map[0] = -1` (index).
3. **Iterate and Accumulate:** Loop through the array, updating the running total with the current element.
4. **Look Backward:** Check the map for `curr_sum - target` (for counting) or check if the exact `curr_sum` has been seen before (for longest subarrays).
5. **Update the Map:** Add the new running total to the map, or update its frequency. (If looking for the *longest* subarray, only store the *first* time you see a prefix).

- **The Minimal Skeleton:**

```
# Skeleton: Counting Subarrays that equal K
def subarray_sum_count(nums, k):
    # freq_map stores {prefix_sum: count_of_occurrences}
    freq_map = {0: 1} # Base case: One way to have sum=0 (empty prefix)

    curr_sum = 0
    total_subarrays = 0

    for num in nums:
        curr_sum += num

        # If (curr_sum - k) exists in map, it means a subarray ending here sums to k
        target = curr_sum - k
        if target in freq_map:
            total_subarrays += freq_map[target]

        # Record the current prefix sum
        freq_map[curr_sum] = freq_map.get(curr_sum, 0) + 1

    return total_subarrays

# Skeleton: Two-Pass Prefix/Suffix (Product Except Self)
def product_except_self(nums):
    n = len(nums)
    res = [1] * n

    # Left pass
    left_prod = 1
    for i in range(n):
        res[i] = left_prod # Assign first
        left_prod *= nums[i] # Then multiply

    # Right pass
    right_prod = 1
    for i in range(n - 1, -1, -1):
        res[i] *= right_prod # Multiply with existing left product
        right_prod *= nums[i]

    return res
```

- **Killer Gotchas:**

- **The Missing Empty Prefix:** Forgetting to initialize the base case (`freq[0] = 1` for counting, or `seen[0] = -1` for indices) is the most common error. Without it, any valid subarray that starts at index 0 will be completely ignored!
- **Overwriting the Map:** When finding the *longest* subarray (e.g., equal 0s and 1s, or multiples of K), you only care about the *earliest* index a prefix sum was seen. Never overwrite an existing entry in the map.
- **Assign Before Multiply:** In a two-pass prefix/suffix array, you must assign the current running product to the result array *before* you multiply the current element into the running product. Otherwise, you include the element itself in the result.
- **1-Indexed Random Picking:** When doing weighted random picks using binary search on a prefix array, pick a random number `t` in `[1, total]`, not `[0, total-1]`. You are finding which bucket the “weight ticket” falls into.

- **Problem Table:** | Problem | Pattern | Key trick | |—|—|—| | p0560 Subarray Sum Equals K | prefix freq map | `freq[curr-k]` count; init `freq[0]=1` | | p0238 Product Except Self | left+right product passes | assign `output[i]` BEFORE multiplying; $O(1)$ extra | | p0525 Contiguous Array | $0 \rightarrow -1$ balance, `first_seen` map | `seen={0:-1}`; same balance at `i,j` $\rightarrow$ equal 0s/1s in `(i,j]` | | p0523 Continuous Subarray Sum | prefix mod, `first_seen` map | `mod_idx={0:-1}`; `gap>1` for `len 2`; never overwrite first index | | p0713 Subarray Product $< K$ | sliding window (not prefix!) | `k<=1 $\rightarrow$ 0`; count $+=$ `right-left+1` subarrays ending at `right` | | p0528 Random Pick with Weight | prefix sums + binary search | `randint(1,total)` (1-indexed); find first prefix `t` |

randomized

- **The Core Intuition (The “Aha!” Moment):** Randomized problems usually fall into two buckets: generating a new uniform distribution from an existing one, or sampling items without replacement. For distributions: Think of rolling two 7-sided dice to pick a number from 1 to 49 by laying out a 7x7 grid. To get 1-10 uniformly, you just accept the first 40 outcomes (which neatly divide into four sets of 1-10) and reject the rest, re-rolling if necessary. For sampling without replacement: Think of a deck of cards. Instead of deleting a drawn card (which is slow), you swap the drawn card with the last card in the deck, then pretend the deck is one card smaller. If the deck is massive (like a 10,000 x 10,000 matrix), you don’t even create the deck; you just use a dictionary to remember the swaps.
- **Signal Recognition (When to use it):**
 - “Implement `randX()` using `randY()`”.
 - “Pick a random element without replacement.”
 - “Uniformly random” or “equal probability.”
 - Massive grids or ranges where generating an actual array or matrix would cause memory errors.
- **The Logical Blueprint (The Universal Steps): For Rejection Sampling (RandX from RandY):**
 1. Create a uniform 2D grid: `(randY() - 1) * Y + randY()`.
 2. Find the largest multiple of X that is $\leq Y * Y$. This is your acceptance threshold.
 3. If your roll falls within the threshold, map it to $1..X$ using modulo arithmetic.
 4. If it falls outside (the “overhang”), reject and re-roll.

For Virtual Fisher-Yates (Massive array sampling):

1. Keep track of the current logical size of the array (`total`).
2. Pick a random index `r` between 0 and `total - 1`.

3. Look up `r` in your dictionary. If it's not there, the value is just `r`.
4. Do the same for the `last` index (`total - 1`).
5. Swap: Set `dictionary[r]` to the value of the `last` index.
6. Delete the `last` index from the dictionary (if it's there) to save space, but *only* if `r != last`.
7. Decrement `total`.

- **The Minimal Skeleton:**

```
# Rejection Sampling (e.g., rand10 from rand7)
def rand10():
    while True:
        # 1. Roll twice to create a uniform 1..49 grid
        row, col = rand7(), rand7()
        idx = (row - 1) * 7 + col

        # 2. Accept up to the largest multiple of 10 (which is 40)
        if idx <= 40:
            return (idx - 1) % 10 + 1
        # 3. Reject 41..49 and loop to re-roll

# Virtual Fisher-Yates
class RandomPicker:
    def __init__(self, total_elements):
        self.total = total_elements
        self.mapping = {}

    def pick(self):
        # 1. Pick a random index in the current valid range
        r = random.randint(0, self.total - 1)

        # 2. Get the actual value mapped to r (or r itself if unmapped)
        res = self.mapping.get(r, r)

        # 3. Get the value of the last valid element
        last_idx = self.total - 1
        last_val = self.mapping.get(last_idx, last_idx)

        # 4. Map r to the last element's value (the swap)
        self.mapping[r] = last_val

        # 5. Cleanup the last element's mapping (unless we picked it)
        if last_idx in self.mapping and last_idx != r:
            del self.mapping[last_idx]

        # 6. Shrink the range
        self.total -= 1
        return res
```

- **Killer Gotchas:**

- **Adding instead of gridding:** `(rand7() + rand7()) % 10` is heavily biased toward middle numbers. Always use the 2D grid formula `(row - 1) * Y + col`.
- **Wrong acceptance threshold:** For `rand10` from `rand7`, you must accept exactly up to

- 40. Accepting up to 42 makes 1 and 2 appear more often. The threshold must be a multiple of the target.
- **Virtual Fisher-Yates cleanup bug:** Deleting `mapping[last]` blindly without checking if `last != r` will delete the new mapping you *just* established if `r` happened to randomly equal `last`!
- **Decoding 1D to 2D:** When mapping a flat index `x` back to a matrix, it's `row = x // cols`, `col = x % cols`. Don't use `rows` instead of `cols` for the modulo!
- **Judge determinism:** Both problem and solution files might need `random.seed(42)` to ensure output matches the WASM judge.
- **Problem Table:** | Problem | Key trick | Essence | |---|---| | p470 Rand10 from Rand7 | 7×7 grid → 1..49; accept 40; `(idx-1)%10+1` | Cutoff = largest multiple of target | source range | | p519 Random Flip Matrix | Dict-based lazy swap-with-last; `mapping.get(k, k)` | Virtual Fisher-Yates; `r==last` edge case; flat→(row,col) decode |

stack

- **The Core Intuition (The “Aha!” Moment):** Think of a stack like a pile of plates. You can only put a new plate on top, and you can only take the top plate off. In coding, this means “deal with the most recent thing first.” For brackets: The last open bracket you saw is the first one you need to close. For nested tasks (like decoding a string `3[a2[c]]`): You read `3[a`, then you see another `2[c`. You must put `3[a` on hold (push to stack), finish `2[c` first, and then come back to `3[a` (pop from stack). The stack acts as your “save state” button.
- **Signal Recognition (When to use it):**
 - “Valid parentheses” or “matching pairs.”
 - Nested structures, like decoding strings or parsing expressions.
 - “What was the previous state?” or “Undo the last action.”
 - Finding the “next greater” or “next smaller” element (this uses a monotonic stack).
- **The Logical Blueprint (The Universal Steps):**
 1. Initialize an empty stack (usually a list or array).
 2. Iterate through the elements one by one.
 3. If the element is an “opener” or “start of a new task”, save your current state by pushing it onto the stack.
 4. If the element is a “closer” or “end of a task”, pop the last state from the stack and combine it with your current result.
 5. At the end, check if the stack is empty (e.g., to ensure all brackets were closed).
- **The Minimal Skeleton:**

```
# Bracket Matching
def is_valid(s):
    stack = []
    mapping = {')': '(', ']': '[', '}': '{'}

    for char in s:
        if char in mapping:
            # It's a closing bracket
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        else:
```

```
# It's an opening bracket
stack.append(char)

return len(stack) == 0

# Nested State (e.g., Decode String)
def decode_string(s):
    stack = []
    current_num = 0
    current_str = ""

    for char in s:
        if char.isdigit():
            # Build the number (might be multiple digits)
            current_num = current_num * 10 + int(char)
        elif char == '[':
            # Save the current state before going deeper
            stack.append((current_num, current_str))
            # Reset for the new nested level
            current_num = 0
            current_str = ""
        elif char == ']':
            # Finished this level, combine with the previous state
            repeat, prev_str = stack.pop()
            current_str = prev_str + current_str * repeat
        else:
            # Normal character
            current_str += char

    return current_str
```

- **Killer Gotchas:**

- **Popping an empty stack:** Always check if `not stack` before trying to look at `stack[-1]` or `stack.pop()`.
- **Forgetting the final check:** In bracket problems, don't just return `True` at the end. Return `len(stack) == 0` (or `not stack`) because there might be unclosed openers leftover.
- **Multi-digit numbers:** In string parsing, numbers can be more than one digit (e.g., `12[a]`). Always build the number: `num = num * 10 + int(char)`.
- **Resetting state:** When pushing state onto the stack to start a nested level, remember to reset *both* the current string and the current number variables.

- **Problem Table:** | Problem | Difficulty | Key trick | Essence | |—|—|—|—| | p020 Valid Parentheses | Easy | `close→open` map + two failure modes | Push openers; closer checks `not stack` OR `top mapping[ch]`; return `stack==[]` at end | | p155 Min Stack | Medium | `min_stack` stores running global min | `min_stack.append(min(val, min_stack[-1]))`; both stacks pop together → $O(1)$ `getMin` | | p394 Decode String | Medium | Multi-digit accumulate + reset BOTH vars on [| Push (`curr_num`, `curr_str`) on [; on]: `prefix + curr_str * repeat`; reset num AND str |

top_k_elements

- **The Core Intuition (The “Aha!” Moment):** Think of a heap as a VIP club with a strict capacity limit `k`. If you want to find the **k largest** elements, you keep the club full. Every time

a new element wants to enter, it challenges the *weakest* (smallest) member of the club. If the newcomer is larger, they kick out the smallest member and take their place. To quickly find the weakest member, the VIP club is organized as a **min-heap**. Conversely, to find the **k smallest** elements, the club must kick out the largest members. So it uses a **max-heap**. This strategy prevents you from having to sort the entire line of people.

- **Signal Recognition (When to use it):**

- “Find the k-th largest/smallest/most frequent element.”
- “Top K elements” or “Closest K elements.”
- Streaming data where you need the “top X” at any given time and sorting the whole stream is too slow.

- **The Logical Blueprint (The Universal Steps):**

1. Determine if you need the k largest (use a min-heap) or k smallest (use a max-heap). Python only has min-heaps, so for a max-heap, multiply your values by -1 before pushing them in.
2. Iterate through the elements one by one.
3. Push the current element onto the heap.
4. If the heap’s size exceeds k, immediately pop from the heap. This automatically removes the element you *don’t* want (the smallest in a min-heap, or the largest in a max-heap).
5. After checking all elements, your heap contains exactly the top k elements. The root (`heap[0]`) is the k-th element.

- **The Minimal Skeleton:**

```
import heapq

# Find K Largest Elements (Uses Min-Heap)
def k_largest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)      # Push current element
        if len(heap) > k:
            heapq.heappop(heap)      # Evict the smallest element
    return heap[0]                  # The k-th largest is at the top of the heap

# Find K Smallest / Closest Elements (Uses Max-Heap simulation via negation)
def k_closest(points, k):
    heap = []
    for x, y in points:
        dist = x*x + y*y            # Calculate distance (skip square root)
        heapq.heappush(heap, (-dist, [x, y])) # Negate distance for max-heap
        if len(heap) > k:
            heapq.heappop(heap)      # Evict the farthest element

    # Extract the points from the heap
    return [point for neg_dist, point in heap]
```

- **Killer Gotchas:**

- **Heap direction mismatch:** The most common mistake is using a max-heap for the k largest. Remember the VIP club analogy: to *keep* the largest, you must easily find and *evict* the smallest. So, keeping the largest requires a **min-heap**.
- **Unnecessary sorting:** Don’t sort the whole array if k is small. Sorting is slow, while a heap of size k is fast.

- **Distance calculations:** For spatial problems (like k closest points), never use `math.sqrt()`. It is slow and unnecessary. Just compare `x*x + y*y`.
- **Python's heapq:** Python's `heapq` module only implements min-heaps. For max-heap behavior, you *must* negate the numbers. Don't forget to un-negate them if you need the actual values later.
- **Problem Table:** | Problem | Difficulty | Heap type | Key trick | |---|---|---| | p215 Kth Largest | Medium | min-heap size k | `heapq.nlargest(k, nums)[-1]` or manual push+pop; root = answer | | p347 Top K Frequent | Medium | sort (or heap) | **Counter** → sort by `-freq`; bucket-sort for O(n) follow-up | | p973 K Closest Points | Medium | max-heap size k | negate `dist2` (skip sqrt); push `(-dist2, point)`, pop when `len > k` | | p407 Trapping Rain Water II | Hard | min-heap BFS | Dijkstra from boundary; push `max(wall_h, nbr_h)` to carry water level; needs `visited` grid |

two_heaps

- **The Core Intuition (The “Aha!” Moment):** Imagine you have a massive pile of numbers and you want to instantly know the middle number (the median). If you split the numbers into two equal piles—a “smaller half” and a “larger half”—the median is just the biggest number in the smaller half, or the smallest number in the larger half. To do this fast, use two heaps:
 1. A **Max-Heap** for the smaller half (so the biggest of the smalls is always on top).
 2. A **Min-Heap** for the larger half (so the smallest of the bigs is always on top). When a new number arrives, pass it through the smaller half to “filter” it, then move the largest from the smaller half into the larger half. If the piles get unbalanced, shift one back. The median is always right at the top!
- **Signal Recognition (When to use it):**
 - “Find the median of a data stream.”
 - “Sliding window median.”
 - Problems where you need to dynamically divide numbers into two halves and check the boundary between them.
- **The Logical Blueprint (The Universal Steps): For Stream Median:**
 1. Create `small` (Max-Heap) and `large` (Min-Heap).
 2. When a number arrives, always push it to `small` first.
 3. Immediately pop the top of `small` and push it to `large`. (This guarantees that the largest item in `small` is correctly placed in `large`).
 4. If `large` now has more elements than `small`, pop the top of `large` and push it back to `small`. (This keeps the rule: `small` always has the same number of elements as `large`, or exactly one more).
 5. The median is the top of `small` (if odd total) or the average of the tops of both heaps (if even total).

For Sliding Window Median (Lazy Deletion): When a number leaves the sliding window, finding it inside a heap to delete it is slow. Instead, use **Lazy Deletion**:

1. Store the index along with the value in the heap: `(value, index)`.
 2. When an element falls out of the window, don't delete it from the heap. Just record its index in a `delayed` dictionary and decrement your *logical* size counters.
 3. Whenever you need to look at the top of a heap, check if its index is in the `delayed` dictionary. If it is, *now* you pop and throw it away (pruning). Keep pruning until the top is a valid, active element.
- **The Minimal Skeleton:**

```
import heapq

# Median of a Data Stream
class MedianFinder:
    def __init__(self):
        self.small = [] # Max-Heap (stores smaller half, negate values)
        self.large = [] # Min-Heap (stores larger half)

    def addNum(self, num):
        # 1. Push to small (max-heap)
        heapq.heappush(self.small, -num)

        # 2. Pop from small and push to large (min-heap)
        largest_in_small = -heapq.heappop(self.small)
        heapq.heappush(self.large, largest_in_small)

        # 3. Balance: small should always be >= large in size
        if len(self.large) > len(self.small):
            smallest_in_large = heapq.heappop(self.large)
            heapq.heappush(self.small, -smallest_in_large)

    def findMedian(self):
        if len(self.small) > len(self.large):
            return -self.small[0] # Odd total: small has the extra element
        # Even total: average the two tops
        return (-self.small[0] + self.large[0]) / 2.0
```

- **Killer Gotchas:**

- **Breaking the rule:** Never try to be smart by writing `if num < small[0]: push to small else: push to large`. Just push to `small`, then pop to `large`. It handles all edge cases automatically.
- **Lazy deletion index requirement:** In sliding window median, you *must* push (`value`, `index`) into the heaps. If you only push values, you cannot tell the difference between two identical numbers when one expires and the other is still valid.
- **Pruning at the wrong time:** In lazy deletion, the *only* thing that matters is the top of the heap. You don't need to prune stale elements buried deep inside. But you *must* proactively prune the top before asking for the median, and before checking sizes.

- **Problem Table:** | Problem | Difficulty | Key trick | Essence | |—|—|—|—| | p295 Median Finder | Hard | push→move→rebalance | Max-heap lower half + min-heap upper half; invariant: `\small\` | `\large, large+1\` | | p480 Sliding Window Median | Hard | lazy deletion via index tuples + `in_small` size tracking | Two heaps + `delayed` dict; decouple logical size from heap physical size | | p355 Design Twitter | Medium | negative timestamp for max-heap ordering | Dump all followee tweets into heap; negate timestamp to get newest-first order |

Tier 3 — Advanced

backtracking

- **The Core Intuition (The “Aha!” Moment):** Imagine you are walking through a large maze. You don't try every path blindly. When you hit a dead end or break a rule, you stop. Then, you

take one step back (backtrack) and try a different way. This saves time because you stop early when a path is bad.

- **Signal Recognition (When to use it):** Look for phrases like “find all valid combinations”, “return true if possible to partition”, “matchsticks to square”, or “solve the board”. The input size N is usually very small ($N \leq 20$), and you are looking for paths that satisfy a strict constraint.
- **The Logical Blueprint (The Universal Steps):**
 1. Sort the input if possible (crucial for cutting off bad branches early).
 2. Create a recursive function that tracks the current state (e.g., remaining budget, current path).
 3. Base case: If the goal is met (budget is 0, board is solved), save a copy of the path and return.
 4. Loop through the possible choices at the current step.
 5. Prune: If a choice is obviously invalid (too large), **break** or **continue**.
 6. Choose: Add the choice to the path.
 7. Explore: Recursively call the function with updated state.
 8. Un-choose: Remove the choice from the path before the next iteration.
- **The Minimal Skeleton:**

```
def solve(candidates, target):
    result = []
    candidates.sort() # Sorting is the key to early pruning

    def backtrack(start: int, remaining: int, path: list[int]):
        if remaining == 0:
            result.append(path[:]) # Always append a copy!
            return

        for i in range(start, len(candidates)):
            # Prune: If this candidate is too big, all subsequent ones are too
            if candidates[i] > remaining:
                break

            # Choose
            path.append(candidates[i])

            # Explore (use `i` to allow reuse, `i + 1` to forbid reuse)
            backtrack(i, remaining - candidates[i], path)

            # Un-choose
            path.pop()

    backtrack(0, target, [])
    return result
```

- **Killer Gotchas:**
 1. **Forgetting `path.pop()`:** This leaks state into sibling branches and corrupts the entire search. Always clean up!
 2. **Appending `path` instead of `path[:]`:** You must append a shallow copy of the path. Otherwise, Python will just store a reference, and your final result will be full of empty lists.
 3. **Pruning unsorted arrays:** You can only **break** early if the array is sorted. If it's unsorted, you might skip a smaller valid number that appears later.

- 4. **Sort descending for bucket-filling:** When partitioning (like filling squares), try the largest items first. They cause early overflows, pruning the tree massively right at the top.
- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p017 Letter Combinations | Digit-to-letters combinations | Standard DFS; one char added per recursion depth | | p039 Combination Sum | Combinations with unlimited reuse | Sort candidates + break loop when `num > remaining`; reuse index `i` in recursive call | | p078 Subsets | Power set of unique elements | Append `path[:]` to result at the **start** of each recursive call (records all prefixes) | | p473 Matchsticks to Square | Partition array into 4 equal sum subsets | Sort descending, backtrack by placing matchstick `idx` into 1 of 4 sides | | p491 Non-decreasing Subseqs | Enumerate non-decreasing subsequences | Keep input order; use a local `used = set()` at each level to dedup sibling calls | | p488 Zuma Game | Min balls to clear board | DFS + memoization on `(collapsed_board, hand_tuple)` |

cyclic_sort

- **The Core Intuition (The “Aha!” Moment):** You have an array of N numbers, and the numbers are from 1 to N . This means each number has its own “correct” index. It is like putting numbered balls into numbered boxes. If ball #3 is in box #5, you just swap it with the ball in box #3. If you cannot move the balls, you can mark box #3 with a negative sign to remember that “ball #3 exists”.
- **Signal Recognition (When to use it):** An array of size N containing numbers in the range $[0, N]$ or $[1, N]$. You’re asked to find missing, duplicate, or disappeared numbers. The constraints demand $O(N)$ time and $O(1)$ space.
- **The Logical Blueprint (The Universal Steps):**
 - **Strategy 1 (Swap into place - Range $[0, N]$):**
 1. Iterate through the array with a while loop.
 2. Check where the current number *should* live.
 3. If it’s not where it belongs (and is within bounds), swap it with the occupant of its correct home.
 4. Otherwise, advance your pointer.
 5. After sorting, do a final scan. The first index that doesn’t match its value is the missing one.
 - **Strategy 2 (Negation Mark - Range $[1, N]$):**
 1. Iterate through the array.
 2. For each number, calculate its target index (e.g., `abs(num) - 1`).
 3. If the value at that index is already negative, you’ve found a duplicate!
 4. Otherwise, flip the value at that index to negative to mark it as “seen”.
 5. A second pass reveals disappeared numbers (their corresponding indices will still be positive).
- **The Minimal Skeleton:**

```
# Strategy 1: Cyclic Swap (Range [0, N])
i = 0
n = len(nums)
while i < n:
    correct_idx = nums[i]
    # Guard against out-of-bounds (the number N has no home) and check if swap is needed
    if correct_idx < n and nums[i] != nums[correct_idx]:
        nums[i], nums[correct_idx] = nums[correct_idx], nums[i]
    else:
        i += 1
```

```
# First index with a mismatch is the answer
for i in range(n):
    if nums[i] != i:
        return i
return n

# Strategy 2: Negation Mark (Range [1, N])
duplicates = []
for num in nums:
    idx = abs(num) - 1 # Use abs() because it might have been negated!
    if nums[idx] < 0:
        duplicates.append(abs(num)) # Already negative -> duplicate
    else:
        nums[idx] = -nums[idx] # Mark as seen

# Find disappeared (indices that remained positive)
disappeared = [i + 1 for i in range(len(nums)) if nums[i] > 0]
```

- **Killer Gotchas:**

1. **The Out-of-Bounds Swap:** In range $[0, N]$, the value N will crash your code if you try to swap it to $nums[N]$. Always guard with `correct_idx < n`.
2. **Advance pointer ONLY in the else block:** If you just swap and immediately do `i += 1`, you skip checking the new number that just landed in `i`.
3. **Forgetting `abs()` in Negation Mark:** Once you start flipping signs, the raw values are corrupted. You *must* use `abs(num)` to compute indices and when appending to results.
4. **Confusing the post-pass scan:** Finding duplicates means checking if the target is *already* negative mid-pass. Finding disappeared numbers means checking what is *still* positive after the full pass.

- **Problem Table:** | Problem | Range | Algorithm | Key trick | |---|---|---| | p268 Missing Number | $[0, n]$ | Cyclic-sort swap | Guard `correct < n`; after sort, first `nums[i]` $i \rightarrow$ answer; fallback `n` | | p442 Find Duplicates | $[1, n]$ | Negation-mark | `idx=abs(num)-1`; already-negative $\rightarrow$ append `abs(num)` | | p448 Find Disappeared | $[1, n]$ | Negation-mark | Same negate pass; collect `i+1` where `nums[i] > 0` after pass |

modified_binary_search

- **The Core Intuition (The “Aha!” Moment):** Normal binary search needs a perfectly sorted array. But the real goal is just to cut the search area in half. You can do this if you can answer a simple yes/no question. For a rotated array, the question is: “Which half is fully sorted?”. For other problems, the question is: “Is this answer possible?”. Instead of searching the array, you search the range of possible answers.
- **Signal Recognition (When to use it):**
 - Arrays that are *rotated* but otherwise sorted.
 - Problems asking to “minimize the maximum” or “maximize the minimum” (e.g., Koko eating bananas, split array largest sum).
 - You need $O(\log N)$ or $O(N \log M)$ performance.
- **The Logical Blueprint (The Universal Steps):**
 - **Pattern A (Rotated Array):**
 1. Find the mid point. Check if it’s the target.
 2. Determine which half is perfectly sorted by comparing `nums[lo]` and `nums[mid]`.

3. If the left is sorted, check if the target falls within that strict range. If yes, go left. If no, go right.
4. If the right is sorted, do the mirror check.
- **Pattern B (Answer-Space / “Minimize the Max”):**
 1. Define the absolute minimum and maximum possible answers (`lo` and `hi`).
 2. Create a monotonic `feasible(mid)` function that tests if a candidate answer works.
 3. Loop `while lo < hi`.
 4. If `mid` is feasible, it might be the answer, or we could do better. Discard the right half by setting `hi = mid`.
 5. If `mid` is not feasible, we *must* go higher. Discard the left half by setting `lo = mid + 1`.

- **The Minimal Skeleton:**

```
# Pattern A: Rotated Array
lo, hi = 0, len(nums) - 1
while lo <= hi: # Target might not exist
    mid = lo + (hi - lo) // 2
    if nums[mid] == target:
        return mid

    if nums[lo] <= nums[mid]: # Left half is sorted
        if nums[lo] <= target < nums[mid]:
            hi = mid - 1      # Target is in the sorted left half
        else:
            lo = mid + 1      # Target must be in the right half
    else: # Right half is sorted
        if nums[mid] < target <= nums[hi]:
            lo = mid + 1      # Target is in the sorted right half
        else:
            hi = mid - 1      # Target must be in the left half
return -1

# Pattern B: Answer Space
lo, hi = min_possible, max_possible
while lo < hi: # We are narrowing down to exactly one answer
    mid = lo + (hi - lo) // 2
    if feasible(mid):
        hi = mid             # It works, but can we go smaller? Keep mid!
    else:
        lo = mid + 1         # Too small, must increase. Discard mid!
return lo
```

- **Killer Gotchas:**

1. **Loop condition mix-ups:** Pattern A uses `while lo <= hi` because we are searching for a specific index. Pattern B uses `while lo < hi` because we are squeezing bounds until they converge on the optimal value.
2. **Discarding valid answers:** In Pattern B, if `feasible(mid)` is true, you must do `hi = mid`. If you do `hi = mid - 1`, you might throw away the correct optimal answer.
3. **The `<=` check for sorted halves:** In Pattern A, use `nums[lo] <= nums[mid]`. The `=` handles the edge case of a 2-element subarray where `lo` and `mid` point to the exact same element.
4. **The Feasibility Function must be Monotonic:** For Pattern B to work, once an answer is feasible, all larger answers must also be feasible (or vice-versa for maximizing). If it fluctuates, binary search will fail.

- **Problem Table:** | Problem | Pattern | Answer range | Key trick | |—|—|—|—| | p033 Search Rotated Sorted Array | A — rotated | index [0, n-1] | `nums[lo] <= nums[mid]` → left sorted; check if target inside [lo, mid), else go right | | p875 Koko Eating Bananas | B — minimize | [1, max(piles)] | `feasible(k): sum(pile/k)` h; as k grows, hours decrease → monotonic | | p410 Split Array Largest Sum | B — minimize | [max(nums), sum(nums)] | `feasible(cap):` greedy pack into subarrays cap; count subarrays k |

subsets

- **The Core Intuition (The “Aha!” Moment):** This pattern is for listing everything. Imagine you have a menu of ingredients, and you want to write down every possible sandwich you can make. You are not looking for one “correct” sandwich. You want to see all combinations. You build them step by step and save every single one.
- **Signal Recognition (When to use it):** The prompt explicitly asks to return **all** subsets, combinations, or permutations. The constraint N is tiny (usually $N \leq 20$). There is no phrasing about “maximizing”, “minimizing”, or finding a specific target sum.
- **The Logical Blueprint (The Universal Steps):**
 1. Use a recursive function to build the combinations choice by choice.
 2. At the start of the function (or when a path reaches the desired length), append a copy of the current path to the results.
 3. Loop through the remaining available choices.
 4. To avoid reusing elements in subsets/combinations, pass a **start** index into the next recursive call, advancing it each time.
 5. For permutations, you either use a **used** set to track what’s currently in the path, or iteratively insert the new number into every slot of existing permutations.
 6. Add to the path, recurse, then pop from the path.
- **The Minimal Skeleton:**

```
# SKELETON: Subsets / Combinations
def subsets(nums):
    result = []

    def backtrack(start, path):
        # Subsets: Record EVERY step of the journey
        result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path) # i + 1 ensures we move forward, preventing reuse
            path.pop()

    backtrack(0, [])
    return result

# SKELETON: Permutations (Iterative BFS Insertion)
def permute(nums):
    perms = [[]]
    for num in nums:
        new_perms = []
        for p in perms:
            # Insert the new number into every possible gap
```

```
    for i in range(len(p) + 1):
        new_perms.append(p[:i] + [num] + p[i:])
    perms = new_perms
    return perms
```

- **Killer Gotchas:**

1. **Subsets with duplicates (p090):** If the input array has duplicates, you will generate duplicate subsets. Fix: Sort the array first, then inside the loop add `if i > start and nums[i] == nums[i-1]: continue`. The `i > start` is vital—it dedupes siblings at the *same* level of the tree, but allows the same value to be used deeper in the tree.
2. **Combinations vs. Subsets:** Subsets append the path to the result on *every single recursive call*. Combinations only append when `len(path) == k`.
3. **Backtracking vs. Subsets Philosophy:** They look identical in code, but subsets is meant to just blindly gather everything, while backtracking has a goal and prunes bad branches heavily.

- **Problem Table:** | Problem | Essence | Key Trick | |—|—|—| | p046 Permutations | All orderings of distinct ints | Iterative insertion: insert new num at every position of each existing perm | | p077 Combinations | All k-sized subsets from [1..n] | **start** index prevents reuse; record only at `len(path)==k`; can prune loop when `n-i+1 < k-len(path)` | | p090 Subsets II | Power set with duplicate elements | Sort + per-level dedup: `if i > start and nums[i] == nums[i-1]: continue` |

trie

- **The Core Intuition (The “Aha!” Moment):** A Trie is a tree for words. It saves space by sharing common starting letters. For example, “car”, “cat”, and “cart” all start with “ca”. The tree stores “c”, then “a”, then splits to “r” and “t”. This makes searching for words very fast. Instead of checking every word in a dictionary, you only follow the letters down the tree.
- **Signal Recognition (When to use it):** Problems asking for autocomplete, prefix matching, word search on a grid (Boggle), dictionary matching with wildcards (e.g., `.at`), or finding words made of other words. You have many strings and repeated prefix queries.
- **The Logical Blueprint (The Universal Steps):**
 1. Define a `TrieNode` containing a dictionary of `children` and an `is_end` boolean (or a `word` string for grid searches).
 2. Initialize the `Trie` with a root node.
 3. **Insert:** Iterate through each character of the word. If the char isn’t in the current node’s `children`, create a new node. Move to the child. After the loop, mark the final node as the end of a word.
 4. **Search:** Walk down the tree char by char. If a char is missing, return false. If you reach the end of the word, check if `is_end` is true.
 5. **Prefix Match:** Same as search, but just return true if you successfully walk down the tree without hitting a dead end.
- **The Minimal Skeleton:**

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
        # self.word = None  <-- Use this instead of is_end for grid searches!
```

```
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def _find(self, prefix: str) -> TrieNode | None:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word: str) -> bool:
        node = self._find(word)
        return node is not None and node.is_end # Must check is_end!

    def starts_with(self, prefix: str) -> bool:
        return self._find(prefix) is not None
```

- **Killer Gotchas:**

1. **search vs starts_with:** `search("app")` must check `node.is_end`. If you inserted "apple", `search("app")` should be false, but `starts_with("app")` should be true. Do not forget the `is_end` check!
2. **Grid DFS (Word Search II) TLE Traps:** You must prune the Trie dynamically! When you find a word, set `node.word = None` to avoid duplicates. More importantly, if a node's `children` dict becomes empty, pop it from its parent to shrink the Trie and avoid re-exploring dead branches.
3. **Word Search II parameter:** Pass the **parent** node into your DFS, not the **current** node. This gives you the reference needed to **pop** dead branches from the tree.
4. **Concatenated Words false positives:** If you insert all words into the Trie *before* checking, a word will just match itself. Sort the array by length, and always check if a word can be concatenated *before* inserting it into the Trie.

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p208 Implement Trie | Foundation: insert/search/starts_with | `_find()` helper shared by both; **search** checks `node.is_end`, **starts_with** checks reachability only | | p211 Design Add/Search Words | Trie + wildcard . | DFS with index `_dfs(node, word, idx)`; at ., iterate all `node.children.values()` | | p212 Word Search II | Build trie from words, DFS board along trie edges | Store **word** at leaf (not `is_end`); in-place '#' marking; prune: null word + pop empty children | | p472 Concatenated Words | Trie + recursive DFS to split word into 2 pieces | Sort by length, insert word **AFTER** check; count `>= 1` at end = 2 pieces total |

Tier 4 — Expert

graph

- **The Core Intuition (The “Aha!” Moment):** A graph connects things together. For ordering problems, think of a to-do list where some tasks must be done before others. If Task A needs Task B, and Task B needs Task A, you are stuck in a circle (a cycle). For “special node” problems, think of an election. You just count who gets the most votes. You do not need to walk through the graph.
- **Signal Recognition (When to use it):**
 - “Prerequisites”, “dependencies”, or “valid ordering” → Use **Topological Sort** (Kahn’s BFS).
 - “Trusted by all but trusts nobody”, “find the center” → Use **Net Degree Score** (simple counting).
- **The Logical Blueprint (The Universal Steps):**
 - **Topological Sort (Kahn’s BFS):**
 1. Count how many prerequisites each node has (**in_degree**).
 2. Build an adjacency list mapping each node to its dependents.
 3. Put all nodes with 0 prerequisites into a queue (they are ready to process).
 4. While the queue isn’t empty, pop a node, add it to your order, and reduce the prerequisite count for all its dependents.
 5. If a dependent’s count hits 0, add it to the queue.
 6. If the final order doesn’t include all nodes, a cycle exists.
 - **Net Degree Score:**
 1. Create an array to track the “score” of each node.
 2. For every directed edge $A \rightarrow B$, decrement A’s score (trusts someone) and increment B’s score (is trusted).
 3. Find the node with a score exactly equal to $N - 1$.
- **The Minimal Skeleton:**

```
# --- Topological Sort (Kahn's BFS) ---
from collections import deque

def topo_sort(n, edges):
    graph = {i: [] for i in range(n)}
    in_degree = [0] * n
    for dest, src in edges:
        graph[src].append(dest)
        in_degree[dest] += 1

    queue = deque(i for i in range(n) if in_degree[i] == 0)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            in_degree[nei] -= 1
            if in_degree[nei] == 0:
```



```
        queue.append(nei)

    return order if len(order) == n else [] # Empty if cycle exists

# --- Net Degree Score ---
def find_special_node(n, edges):
    scores = [0] * (n + 1) # Assuming 1-indexed nodes
    for u, v in edges:
        scores[u] -= 1      # u points out
        scores[v] += 1      # v is pointed to

    for i in range(1, n + 1):
        if scores[i] == n - 1:
            return i
    return -1
```

- **Killer Gotchas:**

1. **Edge Direction:** Add edge `prereq -> course`. If you reverse this, your zero-in-degree nodes will be completely wrong.
2. **1-Indexed Nodes:** Many degree problems (like town judge) use nodes labeled 1 to N. Array size must be `N + 1`, and loops must iterate `range(1, N + 1)`.
3. **Cycle Detection Check:** Always check if `len(order) == n` at the end of Kahn's. If nodes are trapped in a cycle, their in-degree never reaches 0, so they are never added to the queue.
4. **Target Score:** The special node is trusted by everyone *else*, so its score must be `N - 1`, not N.

- **Problem Table:** | Problem | Difficulty | Essence | Key trick | |---|---|---| | p207 Course Schedule | Medium | Can all courses be taken? (cycle detection) | Kahn's BFS; `len(order)==n` → True | | p210 Course Schedule II | Medium | Return a valid course order, [] if cycle | Same Kahn's; collect `order`, return [] on cycle | | p997 Find the Town Judge | Easy | Find node trusted by everyone, trusts nobody | Net score array `[0]*(n+1)`; judge score == `n-1` |

greedy

- **The Core Intuition (The “Aha!” Moment):** Always make the best choice right now. Do not worry about the future steps. For example, if you want to use the fewest arrows to pop balloons, shoot the arrow where the most balloons overlap right now. If making the best choice now does not ruin future choices, you can just move forward. You only need to keep track of the best state so far. You never need to go back.
- **Signal Recognition (When to use it):**
 - “Can you reach the end?”, “Minimum number of jumps/arrows”, “Maximum profit”.
 - Finding a valid starting point in a circular route (Gas Station).
 - Interval problems where you want to minimize overlap or maximize non-overlapping intervals.
- **The Logical Blueprint (The Universal Steps):**
 - **Intervals:** Sort by the *end* time. Track the end of the current selection. Iterate through; if a new item starts *after* your tracked end, you must make a new selection (e.g., shoot another arrow) and update the tracked end.

- **Running Surplus (Gas Station):** Track a global surplus and a local running surplus. If the local surplus drops below zero, the current starting point is invalid; reset the starting point to the very next item and reset local surplus. At the end, if global surplus ≥ 0 , return the start point.
- **Max Reach (Jump Game):** Track the furthest index you can reach. Iterate through indices. At each step, update the furthest reach. If your current index exceeds your furthest reach, you're stuck.

- **The Minimal Skeleton:**

```
# --- Interval Selection (e.g., Min Arrows) ---
def greedy_intervals(intervals):
    if not intervals: return 0
    intervals.sort(key=lambda x: x[1]) # Sort by END coordinate
    count = 1
    end = intervals[0][1]

    for start, finish in intervals[1:]:
        if start > end:                # No overlap
            count += 1
            end = finish                # Update frontier
    return count

# --- Running Surplus (e.g., Gas Station) ---
def greedy_surplus(gain, cost):
    total_surplus = local_surplus = start = 0
    for i in range(len(gain)):
        net = gain[i] - cost[i]
        total_surplus += net
        local_surplus += net

        if local_surplus < 0:          # Failed here
            start = i + 1              # Try next index
            local_surplus = 0          # Reset local

    return start if total_surplus >= 0 else -1
```

- **Killer Gotchas:**

1. **Sort Intervals by End:** Sorting by start fails for nested intervals (e.g., [1, 10] and [2, 3]). You must sort by *end* point to greedily process intervals that finish earliest.
2. **Global Validation:** For circular surplus problems, resetting *start* locally isn't enough. You **MUST** check that *total_surplus* ≥ 0 at the end to ensure the journey is globally possible.
3. **Task Scheduler Math:** The math formula $(\text{max_freq} - 1) * (n + 1) + \text{max_count}$ dictates the length. The -1 is crucial because the final execution of the most frequent task requires no trailing cooldown.

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p055 Jump Game | Farthest reachable index | Update *max_reach* = $\max(\text{max_reach}, i + \text{nums}[i])$; return False if $i > \text{max_reach}$ | | p134 Gas Station | Find circular starting point | Reset start to $i+1$ when *current_surplus* < 0 ; return *start* if *total_surplus* ≥ 0 else -1 | | p135 Candy | Distribute candies based on ratings | Two passes: $L \rightarrow R$, then $R \rightarrow L$ taking $\max(\text{candies}[i], \text{candies}[i+1]+1)$ | | p452 Arrows to Burst Balloons | Min arrows to pop all intervals | Sort

by end coordinate; shoot arrow at `end`, increment when `start > end` | | p455 Assign Cookies | Satisfy children with cookies | Sort both; two-pointer match child greed vs cookie size | | p502 IPO | Maximize capital using at most k projects | Sort by capital; max-heap of profits of currently affordable projects; pop k times | | p621 Task Scheduler | Min CPU cycles with cooldown n | Formula: $\max(\text{len}(\text{tasks}), (\text{max_freq} - 1) * (n + 1) + \text{max_count})$ | | p621b Task Scheduler Multi-Machine | Task scheduler with m machines | Simulation: assign highest-freq task not in cooldown per machine; fast-forward time on deadlock |

matrix_traversal

- **The Core Intuition (The “Aha!” Moment):** A matrix is just a grid of numbers. Do not get confused by complex loops. Find the simple geometric rule:
 - To rotate 90 degrees: swap rows with columns, then reverse each row.
 - To read in a spiral: imagine 4 walls moving inward.
 - To find diagonals: cells on the same diagonal have the same `row + col` sum.
 - To find the shortest distance to water: imagine dropping a stone in all water cells at the same time and watching the waves spread out.
- **Signal Recognition (When to use it):**
 - “Rotate 90 degrees in-place”.
 - “Spiral order” or “layer order”.
 - “Diagonal traverse”.
 - “Nearest 0 for every cell” or “distance to nearest X” (implicitly a grid graph).
- **The Logical Blueprint (The Universal Steps):**
 - **Rotation:** Transpose the matrix (swap `matrix[i][j]` with `matrix[j][i]`), then reverse every row.
 - **Spiral:** Define 4 boundaries (`top`, `bottom`, `left`, `right`). Traverse the top row, then right col, then bottom row (if `top <= bottom`), then left col (if `left <= right`). Shrink the boundaries inward after each segment.
 - **Diagonal:** Group cells into a hash map by `diagonal_id = row + col`. Then read the groups sequentially. Reverse the elements for even IDs (to simulate the zig-zag up-right direction).
 - **Multi-source BFS:** Enqueue all target cells (e.g., all 0s) at distance 0. Mark all other cells as unvisited (-1). Run a standard BFS.
- **The Minimal Skeleton:**

```
# --- Rotation (In-place) ---
def rotate(matrix):
    n = len(matrix)
    # Transpose
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse rows
    for row in matrix:
        row.reverse()

# --- Diagonal Grouping ---
def diagonal_traverse(mat):
```

```
m, n = len(mat), len(mat[0])
diags = {}
for i in range(m):
    for j in range(n):
        diags.setdefault(i + j, []).append(mat[i][j])

result = []
for d in range(m + n - 1):
    if d % 2 == 0:
        result.extend(reversed(diags[d]))
    else:
        result.extend(diags[d])
return result
```

- **Killer Gotchas:**

1. **Spiral Double-Counting:** After traversing the top and right edges and shrinking top and right, you MUST check if top <= bottom and if left <= right before traversing the bottom and left edges. Otherwise, a 1D strip gets counted twice.
2. **Transpose Bounds:** When transposing, the inner loop must be for j in range(i + 1, n). If you start from 0, you swap everything twice, effectively undoing the transpose!
3. **Multi-Source BFS Sentinel:** Initialize distances of non-target cells to -1 instead of using a visited set. Distances are non-negative, so -1 acts as a perfect, collision-free unvisited sentinel.

- **Problem Table:** | Problem | Approach | Key Trick | |---|---| | p048 Rotate Image | Transpose then reverse each row | Transpose swaps (i,j) (j,i) for j>i only; then row.reverse() = clockwise 90° | | p054 Spiral Matrix | 4 boundary pointers, shrink inward | Guard top bottom / left right before bottom-row & left-col loops to avoid double-append | | p498 Diagonal Traverse | Group cells by d=i+j; reverse even d | All cells with same i+j lie on one diagonal; even d → up-right → reverse the list | | p542 01 Matrix | Multi-source BFS from all 0-cells | Seed queue with every 0, mark 1-cells as dist=-1; BFS propagates distance outward |

monotonic_stack

- **The Core Intuition (The “Aha!” Moment):** A monotonic stack helps you remember items that are “waiting” for a future condition. Imagine a line of people. You are looking for the next taller person. A short person waits in the stack. When a tall person arrives, the tall person answers the question for all short people waiting. We then remove the short people from the stack. We store the index, not just the value, so we can calculate the distance or width between them.
- **Signal Recognition (When to use it):**
 - “Next greater/smaller element”.
 - “Distance to the next warmer/colder day”.
 - “Largest rectangle in a histogram”.
 - “Sum of all subarray minimums/maximums” (element contribution).
- **The Logical Blueprint (The Universal Steps):**
 1. Initialize an empty stack and a result array.
 2. Iterate through the array (sometimes appending a sentinel like 0 to force a final flush).
 3. While the stack is not empty and the current element breaks the required order (e.g., current > top for a decreasing stack):

- Pop the top index.
 - The current element is the “right boundary” or “next greater” for the popped element.
 - The new top of the stack is the “left boundary”.
 - Calculate width, area, or answer for the popped element.
4. Push the current index onto the stack.

- **The Minimal Skeleton:**

```
# --- Next Greater Element (Decreasing Stack) ---
def next_greater(nums):
    ans = [-1] * len(nums)
    stack = [] # stores indices
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            popped_idx = stack.pop()
            ans[popped_idx] = num # Resolved!
        stack.append(i)
    return ans

# --- Largest Rectangle (Increasing Stack with Sentinel) ---
def largest_rectangle(heights):
    heights.append(0) # Sentinel to flush the stack at the end
    stack = []
    max_area = 0
    for i, h in enumerate(heights):
        while stack and h < heights[stack[-1]]:
            height = heights[stack.pop()]
            # Width spans from current index to the new stack top
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    heights.pop() # Clean up the sentinel
    return max_area
```

- **Killer Gotchas:**

1. **Width Calculation:** For the largest rectangle, the width for a popped element is $i - \text{stack}[-1] - 1$ (if the stack isn't empty). It is NOT $i - \text{popped_idx}$. The left boundary is defined by the element that *remains* beneath it on the stack.
 2. **The Sentinel:** Always append a 0 (or -1 if values are positive) at the end of the array for histogram/rectangle problems. If you don't, elements forming a rising slope at the end of the array never get popped and processed.
 3. **Duplicate Handling (Subarray Contributions):** When calculating subarray mins/maxes, avoid double-counting subarrays with duplicate values by making one pass strictly greater/less ($>$, $<$) and the other inclusive ($>=$, $<=$).
- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p739 Daily Temperatures | Next warmer day distance | Decreasing stack of indices; $\text{ans}[j] = i - j$ on pop | | p084 Largest Rectangle | Max rectangle in histogram | Increasing stack + sentinel 0; $\text{width} = i - \text{stack}[-1] - 1$ on pop | | p503 Next Greater II | Circular next greater element | Loop $2*n$ with $\text{idx} = i \% n$; only push to stack when $i < n$ | | p456 132 Pattern | Check subsequence $\text{nums}[i] < \text{nums}[k] < \text{nums}[j]$ | Loop backwards; stack tracks j (max), third tracks popped k | | p853 Car Fleet | Number of fleets reaching target | Sort pos descending; count times where $\text{time} > \text{slowest_time}$

| | p907 Sum of Subarray Mins | Subarray minimums contribution | Two passes (left < and right bounds); contribution = val * left_gap * right_gap |

union_find

- **The Core Intuition (The “Aha!” Moment):** Think of nodes as people in different groups. Every group has a boss (the root). To check if two people are in the same group, you just find their bosses. If the bosses are the same, they are in the same group (**find**). If they want to merge groups (**union**), you simply make one boss report to the other boss. To make this very fast, use “path compression”: every time a person finds their boss, they save the boss’s number directly. This flattens the tree.
- **Signal Recognition (When to use it):**
 - “Are these connected?”, “Count connected components”.
 - “Find the redundant edge”, “Is there a cycle?” in an UNDIRECTED graph.
 - Satisfiability of equality equations (**==** and **!=**).
 - Dynamic grouping where you frequently add edges and query connectivity.
- **The Logical Blueprint (The Universal Steps):**
 1. Initialize a **parent** array where every node points to itself, and a **rank** (or **size**) array initialized to 0.
 2. Define **find(x)**: Recursively find the root. As you return, update the parent pointer of every node along the path to point directly to the root (Path Compression).
 3. Define **union(x, y)**: Find the roots of **x** and **y**. If they are the same, a cycle is detected. If different, attach the tree with the smaller rank under the root of the tree with the larger rank (Union by Rank).
- **The Minimal Skeleton:**

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            # Path compression
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return False # Already in same set (cycle)

        # Union by rank
        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        elif self.rank[root_x] > self.rank[root_y]:
            self.parent[root_y] = root_x
```

```
else:
    self.parent[root_y] = root_x
    self.rank[root_x] += 1

return True # Successfully merged
```

- **Killer Gotchas:**

1. **1-Indexed Nodes:** Many problems (like Redundant Connection) use nodes labeled 1 to N. Your `parent` array must be initialized to size `N + 1` (`list(range(n + 1))`). Off-by-one errors here will cause out-of-bounds exceptions or silent logic bugs.
2. **Two Passes for Equations:** For equality satisfiability (e.g., `a==b`, `c!=d`), you **MUST** process all `==` equations in the first pass to build the clubs, and then verify all `!=` equations in a second pass. Interleaving them fails.
3. **Cycle Detection = False Return:** If `union(x, y)` finds that `root_x == root_y`, returning `False` is exactly how you detect a redundant edge.
4. **Only Increment Rank on Equality:** `rank` is a rough tree depth. It only increases when you merge two trees of the *exact same rank*. Don't increment it indiscriminately.

- **Problem Table:** | Problem | Essence | Key trick | |—|—|—| | p323 Connected Components | `n - successful_unions` | Decrement counter only when `union()` returns `True` | | p684 Redundant Connection | First edge linking already-connected nodes | Nodes are **1-indexed**; parent size = `n+1`; return on first `False` | | p990 Equality Equations | Union all `==`, then reject `!=` contradictions | Fixed 26-node pool; `eq[1]=='='` to detect `==`; `eq[0]/eq[3]` are the variable chars |